ML Offline Design

Custom Bidding Agent — ML Lane

Sigma Software — Rodrigo Cunha

2026-10-27

Contents

Part I — Design
00 — Overview
- Purpose and scope
- System map
- Goals and models
- Components
- Touchpoint contracts
- Cross-cutting concerns
- Environments
- Open questions

01 — Data access
- What we read
- The account boundary
- Source format
- Read path into Databricks
- Athena's role
- Access control
- Cadence
- Open questions

02 — Datasets
- The layers
- Catalog and naming
- Grain and keys per dataset
- Identity-space naming convention
- Privacy treatment
- Retention and backfill
- Incremental vs. full rebuild
- Open questions

03 — Labels
- The label
- Dataflow
- Inputs
- Label parameters
- Join mechanics
- Maturity
- Output
- Open questions

04 — Feature layer and hot-feature jobs
- The no-skew contract
- What a definition declares
- The two consumers
- Feature materialization
- Repo home

Freshness and cadence
Keyspace management
Feature lifecycle
Planned feature families
Open questions
05 — Training
Model configuration
Pipeline structure
Training-set build
Hyperparameter tuning
Model fit
Calibration fit
Evaluation
Export, parity, registration
Lineage and reproducibility
Open questions
06 — Validation and promotion
Decision inputs
Rollout policy
Retraining modes
Rollout states and readout
A/B test definition
Concurrency during an active A/B
Guardrails and automated abort
Approval gates
Evidence modes
Launch readiness
Open questions
07 — Deployment job
Trigger
Task structure
The writes
Rollback
Where this runs
Execution model
Concurrency and queueing
Open questions
08 — Orchestration and repos
The two repos
Pipelines are submitted, not pre-created
Configs and how they reach their consumers
The deployment job
Environments
CI/CD
Testing
Open questions

09 — Monitoring and decision-log ingestion
Decision-log ingestion
Per-arm readout tables
Model monitoring
Data-quality monitoring
Pipeline alerting
Retraining triggers
Bidder response dataset
Open questions
Part II — Annex
Client Custom Bidding Agent — Discovery Summary
Part 1 — Client-provided facts
1.1 Project overview
1.2 The four services and request flow
1.3 Monolith vs. multi-service benchmark (SW team)
1.4 Data & labels
1.5 Infrastructure & environment
1.6 Compliance
1.7 Non-functional / success criteria (client-stated)
1.8 Process
1.9 Batch-02 client reports (2026-10)
Part 2 — Our analysis & working decisions
2.1 What we own
2.2 Working leans (Discovery Report §2)
2.3 Assumptions until told otherwise (Discovery Report §3)
2.4 Open questions (moved)
2.5 Dependencies & risks
2.6 ADR status
2.7 Testing lane (Olena, 2026-10)
Team roster
Artifacts (current)
Contradictions & gaps noticed
ADR-0000: Record architecture decisions for this project
ADR-0001: ML service returns calibrated P(Action), not expected value
ADR-0002: MVP serving is ONNX tabular GBDT on CPU; Triton/GPU deferred
ADR-0003: Calibration is a versioned matched-pair artifact baked into the served model
ADR-0004: Attribution window and method that defines the training label
ADR-0005: Cross-boundary data-access pattern (curated exports vs. direct Athena)
ADR-0006: Ship preprocessing, model, and calibration as one ONNX file

Part I — Design

00 — Overview

The end-to-end map of the offline ML platform. Every other doc in this set details one box of the diagram below; this doc fixes the boxes, the arrows, and the boundary with serving.

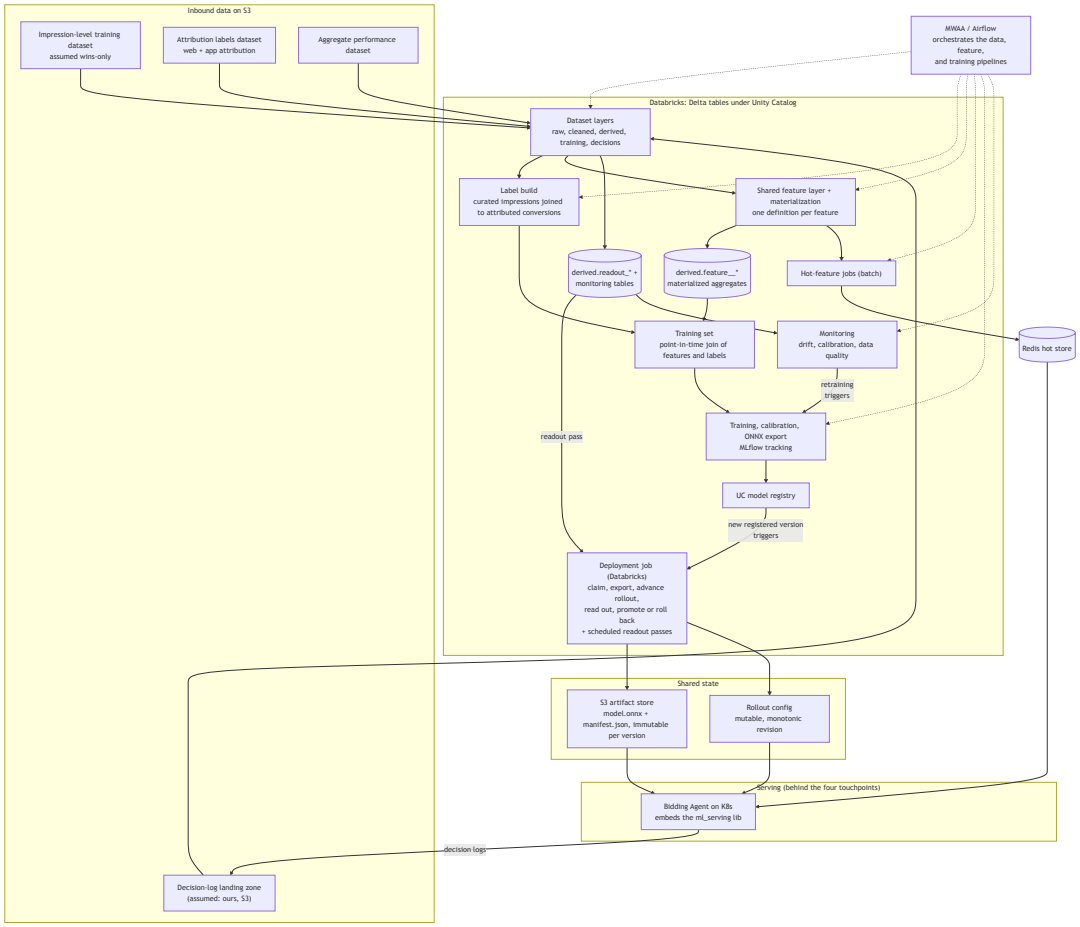
Purpose and scope

The offline platform produces everything the `ml_serving` Go library consumes and consumes everything it emits. The boundary is the four touchpoint contracts in this doc: the published artifact, the rollout config, the hot-feature store, and the decision logs. Offline owns everything up to the first three and everything after the fourth. The Bidding Agent and the lib own the space in between; we never reach into it, and it never reaches into Databricks or MLflow.

One team owns both sides. Responsibilities split per person (Go, ML, data engineering, QA), not per organization, so no contract here crosses a team boundary; they cross repo and lane boundaries only — the Data Engineering lane that owns data provenance (`01_data_access.md`) is such a per-person lane inside the team, not another organization.

Platform: Databricks runs the ML workloads (client-confirmed). Within it, Delta tables, Unity Catalog, and MLflow with the UC model registry for storage, tracking, and registry are our directed picks (`PLAN.md`), not client-mandated; our own MWAA (Airflow) instance for orchestration is likewise directed. Input data is reachable from our account; who curates it — the client directly, or our Data Engineer curating from the client's internal tables — is the Data Engineering lane's scope, unresolved (`adr/0005`), and immaterial to this platform's design. The transfer mechanism is open (`01_data_access.md`).

System map



Solid arrows are data flow; dotted arrows are orchestration. Two loops close through the diagram. The training loop: the client's impression-level dataset (assumed wins-only, so every row is a served impression — a working assumption to confirm, `03_labels.md`) joins to the attribution labels dataset to produce the label and, with the features, the next training set. The decision-log loop: the lib logs every scored candidate, the rows land in our landing zone, and they feed the per-arm readout and monitoring; they are never the label universe, because a scored candidate may have lost the auction. The same rule covers the client's auction and bid data: the label universe is served impressions, because `P(Action | Impression)` conditions on one; auction and bid data enter only as feature and analysis sources. Guardrail breaches on serving health route directly from the observability stack's own alert rules to the deployment job's abort path through the jobs API — never through the monitoring job above (`09_monitoring.md`). One Databricks deployment job, triggered when training registers a new version, owns the whole rollout: each rollout state (fleet load, shadow, canary ramp — A/B nested inside the ramp when a policy enables it) is just a content of the rollout config, and the job advances by writing successive revisions, reading out the per-arm tables between advances. It is the only writer of shared state.

Goals and models

The ad set's `optimization_goal` decides whether a model is consulted and which one. The goal values are the client's proposed enum — both `optimization_goal` and `bidding_strategy` are proposed fields that do not yet exist in the ad set data model. Slot names are ours (lowercase); goal values are the client's spelling:

`optimization_goal`	Bidder behavior	ML model (slot)	Labels from	Status
`impressions`	Deterministic: `P = 1`; `EV = max_bid / 1000` (the max_cpm strategy's formula). ML never called.	none	—	Effectively live today (all ad sets behave as this; the field itself is proposed)
`PIXEL_PAGE_VIEW`	Calls ML for `P(page view)`	`pixel_page_view`	Web pixel via the web-attribution provider (`event_name` mapping pending the client)	Initial
`PIXEL_PURCHASE`	Calls ML for `P(purchase)`	`pixel_purchase`	Web pixel via the web-attribution provider (`event_name` mapping pending the client)	Initial
`app_installs`	Calls ML for `P(install)`	future slot	App events via MMP	Future, out of MVP

`bidding_strategy` never changes the model: `goal_cpa` and `lowest_cost` use the same goal's slot, only the value multiplied downstream differs, and `max_cpm` behaves as `impressions`. Routing is by goal alone. If the client ships advertiser-selectable windows, each offered (goal, window) becomes its own slot (window placement and the modeling approach per window are still open); the client also names further pixel goals (lead generation) as future — each is a new slot, no architecture change.

Components

Each component is owned by one detail doc.

- Dataset layers (`02_datasets.md`) — four stages: raw (Unity Catalog external tables reading the curated datasets in place, no copy), cleaned (the first materialized Delta tables: typed, deduplicated, identity resolved — reshaped the client content, nothing computed), derived (tables whose content we compute: labels, feature tables, readout and monitoring tables), training (immutable point-in-time snapshots, one per run); plus a decisions schema for ingested serving events. Read path in

`01_data_access.md`. Also owns the identity-space naming convention and the privacy drop/hash/retain rules.

- Label build (`03_labels.md`) — joins the client's impression-level training dataset (assumed wins-only) to the attribution labels dataset, bridged by the impression key (assumed `impression_id`; confirming both assumptions is `03_labels.md`'s open question 3). The client exposes 7/14/30-day attribution outputs (the web-attribution provider for web, MMP for app; app goals are out of MVP scope; the client's ML report states a 1-day variant is also available, to confirm), and the window may become advertiser-facing configuration. Both initial goals (Page View, Purchase) get label configs, each on a single window for now; the expected end state is one label config and one model per (goal, window), with the shipped windows pending label-volume data and the client's offered-window set. Labels mature late; recent data is only partially labeled, and the build tracks maturity explicitly. The web-attribution provider's `ad_id` is nullable on the conversion row; it never affects the label join, it only shrinks rollups joining conversions directly to ads or campaigns, tracked as a data-quality metric.
- Shared feature layer (`04_features.md`) — one definition per feature, called by the feature-materialization pipeline (which maintains the historical aggregates incrementally in `derived.feature_<model>_*` tables the training-set build joins point-in-time) and by the hot-feature jobs. This is the no-skew contract: the value a model trained with and the value the serving-time join finds in Redis come from the same code. Proven at dev scale in ml_serving (one shared feature function feeds both the training join and the store seeder; the export gates on an ONNX-versus-Python parity check at 1e-4).
- Hot-feature jobs (`04_features.md`) — batch Databricks jobs on an MWAA schedule that compute rolling-window aggregates and write them to Redis under the key templates declared in the feature definitions (the manifest's `hot_features` section is generated from those same definitions). No streaming component.
- Training (`05_training.md`) — the training-set build (the diagram's "Training set" node: point-in-time join of labels and features, with the maturity cutoff applied) plus training, calibration, and ONNX export as Databricks workloads with MLflow tracking. The export bakes preprocessing, model, and calibration into one graph and gates on an ONNX-versus-Python parity check.
- Deployment job (`06_validation_promotion.md`, `07_deployment_job.md`) — one Databricks job attached to the registered model, triggered when a new version registers, one task per step: claim (sets the challenger alias — training already gated quality, no replay here), export (the artifact pair to an immutable S3 path), advance the rollout (write the next config revision), read out (per-arm tables from decision rows and matured labels), promote or roll back (move the champion and previous aliases; rollback is a config write pointing at a previous version). The advance and readout tasks loop through the rollout states as scheduled passes of the same job, with an alert trigger for immediate aborts; the rollout config's content is the state (`07_deployment_job.md`, Execution model). Two docs, one component: `06` owns the decision logic (gates, metrics, readout, approval placement), `07` owns the job itself (task structure, export and config-write mechanics). Not an MWAA DAG.
- Orchestration (`08_orchestration_repos.md`) — MWAA DAGs submit the data, feature, and training runs to Databricks at trigger time, each pinned to its pipeline's `package_version` (the deployment job is the exception: a standing job triggered by

model registration). Two dedicated repos split definitions from machinery: the DAG repo holds the framework and every per-model definition; the workload repo is the pipeline-agnostic versioned package. A manual backfill DAG re-runs the same workloads over explicit ranges.

- Monitoring (`09_monitoring.md`) — the decision-ingest pipeline and readout-table build (`decisions.scored_candidates`, `derived.readout_<model>`); one monitoring DAG, parametrized per model, computing `derived.model_metrics` and `derived.data_quality` daily and firing the mapped actions (recalibrate-only run, full train run, alerts); the retraining triggers the training pipeline executes; pipeline alerting; and the bidder response dataset the client wants back.

Touchpoint contracts

What serving expects is defined in the `ml_serving` repo (its `docs/SCHEMAS.md` and `testdata/`; the schema drafts are pending team review, and the shapes below are the working contract). This section states the offline-side obligations.

1. Published artifact

Per version, an immutable S3 path `<model_name>/v<version>/` holding `model.onnx` and `manifest.json`. The ONNX graph contains preprocessing, model, and calibration; one input tensor per raw field, named identically to the field; raw strings pass through untouched. The manifest carries `input_schema` (`name`, `source`: request/candidate/hot, `dtype`: numeric/string) and `hot_features` (`name`, `join_keys`, `key_template`, `default`), plus `onnx.file` and `onnx.output_name` (artifact filename and calibrated-probability output are manifest-declared, not hardcoded). Offline obligations: the export parity check (ONNX versus Python on held-out rows) is a hard gate we own; `mlflow_run_id` in the manifest is our lineage pointer, ignored by the lib; published paths are never rewritten.

2. Rollout config

Path: `<model_name>/rollout_config.json`, sibling to that model's `v<version>/` artifact folders. One mutable JSON per model slot: `model_name`, monotonic `revision`, `updated_at` and `updated_by` for audit, `champion` (`version`, `uri`, `checksums` for both files), later `challenger` and `routing`, and later still `queued` (serving-side addition, not yet in the lib's schema; holds at most one version waiting on a busy slot — a newer registration overwrites it, not a FIFO backlog). Slots are named per goal, later per goal and window; the bidding agent groups each request's candidates by goal and calls the lib per slot, so a new goal is a new slot and config file, not an interface change. The lib reacts only to revision changes and fails static. Offline obligations: the deployment job is the only writer; it validates before writing (schema, artifact exists, checksums match); every revision strictly increases; checksums live in the config so the mutable pointer is tied to immutable content.

3. Hot-feature store

A flat Redis keyspace of float scalars, one key per rendered `key_template`. The lib mirrors the whole keyspace every 5–10 minutes and knows nothing about windows or aggregation. Offline obligations: the hot-feature jobs own all windowing logic; each model owns its key-template namespace prefix (the anti-collision contract); values must parse as float64. The freshness a hot feature can promise is floored by three delays in series: how fresh the inbound curated data is when it crosses the account boundary, plus our job cadence, plus the lib's mirror interval; `04_features.md` sizes this.

Everything runs in one installation: the Bidding Agent on Kubernetes, Redis expected to be a managed AWS service reachable by both our jobs and the agent. The exact service choice is deferred.

4. Decision logs

One row per scored candidate. What the lib's `DecisionRecord` carries today: model name, version, arm, config revision, candidate id, probability, timestamp. What we also need and don't have yet (tracked in `OPEN_QUESTIONS.md`'s serving-side changes list): request id, the model inputs, the routing salt, the routing-unit value, and a per-candidate scored-versus-default flag. Rows join impressions and labels by `(request_id, candidate_id)` once request id lands. Offline obligations: ingest the rows into the dataset layers and build the per-arm readout tables the deployment job's readout pass judges. Decision logs also feed monitoring, and are the raw material for the bidder response dataset the client wants back (`09_monitoring.md`); they are never the label universe. The serving side has not settled where the rows land: the lib exposes an async logger interface and its docs name an ingest only as a placeholder. We assume the rows become available on S3 (the landing zone in the diagram); the design holds for any location we can read.

Cross-cutting concerns

- Identity mapping — three ID spaces exist: client UUIDs, DSP IDs, and raw-log integer IDs. The client bridges the first two with `external_id` (a string on the client side; documented as comma-separated for one entity, placements) and `alt_id` (the client UUID stored on the DSP side); raw logs carry integer ids that must be mapped, never read as UUIDs. the client's dataset guidance says curated datasets should expose all three as separate named fields; the client controls the identity mapping (stated), but who fills the triplets in the curated feed is unconfirmed (`02_datasets.md` holds the operative check). Our obligation: every join key in a feature definition declares which of the three spaces it is in, in the definition metadata (`04_features.md`); our table columns carry explicit space suffixes. `02_datasets.md` fixes the naming convention once.
- Label maturity — attributed conversions are expected on a daily cadence (the client's proposed expectation for the curated feed, not a live cadence yet) and windows stay open up to 30 days, so recent impressions are only partially labeled. Training-set construction and promotion gates must account for it; early rollout readouts run on proxy metrics while true labels trail.

- Privacy — input data carries privacy-sensitive fields (`ifa`, IP-type fields, device ids, `hhid`, `user_id`, `user_email`, `ga_client_id`, tracking URLs) and consent signals (`is_gdpr`, `is_gdpr_consented`, `gdpr_consent_string`, `us_privacy`, `dnt`, `lmt`). `user_id` arrives non-null on the web attribution tables backing the labels dataset, so it reaches us whether or not we use it. The dataset layers (`02_datasets.md`) define which fields are dropped, hashed, or retained, and under what retention.

Environments

Settled: two environments, dev and prod; separate S3 prefixes for artifacts and rollout configs per environment; no staging environment. Working assumptions, open until the account and MWAA questions land: both environments in the same AWS account, one Databricks workspace to start (separated by Unity Catalog catalog, `ml_dev` / `ml_prod`, with permissions per catalog), and possibly one MWAA instance per environment. `08_orchestration_repos.md` encodes the mechanics and tracks both questions.

Open questions

1. Which attribution windows the client will offer per goal (possibly advertiser-selectable), and label volumes per (goal, window). Both initial goals (Page View, Purchase) ship, so the volumes decide each goal's windows, not which goal goes first. The attribution method (click- versus view-through) is a separate question: the client's tables do not record it, so the question to the client is whether that information can be exposed at all (`03_labels.md`).
2. The concrete decision-log sink, a serving-side decision not yet made (see touchpoint 4). We assume the rows land on S3.

01 — Data access

How we read the input data `00_overview.md` assumes is already on S3, in an account we can read. How it got there is out of scope; this doc is the read path from that data into Databricks, and nothing before it.

What we read

The five curated datasets below, taken as a given. The client's stated design principle is that the bidder consumes curated outputs prepared on a defined cadence, not raw internal production tables; whether the client delivers them directly or our team's Data Engineer curates them from the client's internal tables is the Data Engineering lane's scope and immaterial to this platform. This doc states what we consume and what we require of it; provenance, curation mechanics, and the boundary mechanism are recorded here only as reference for the DE lane.

The client has proposed, not yet finalized, a five-dataset split for what crosses the account boundary:

- Delivery-control feed — low-latency campaign, ad set, ad, creative, budget, status, targeting, and override state. The pacing and ingestor lanes' input; not ours directly.
- Impression-level training dataset — joined delivery, campaign, ad set, ad, creative, supply, bid, win, spend, and event outcome data (the client's wording). Our working assumption, to confirm: it is wins-only, every row a served impression — the client's description does not say so. This is the training and label universe, our primary input (`03_labels.md` open question 3).
- Attribution labels dataset — conversion and attribution outputs joined back to impressions, ads, ad sets, campaigns, and attribution windows. Feeds `03_labels.md` directly.
- Aggregate performance dataset — hourly or daily rollups for pacing, reporting validation, and spend reconciliation. A secondary input, used for data-quality checks against the other two.
- Bidder response dataset — outbound: our decisions and model outputs, sent back to the client. Not an input to us. The client stated the need (visibility into how the bidder decided) and a candidate field list (auction/request IDs, the client and DSP IDs, bid/no-bid decision, bid price, model version, decision timestamp, latency, no-bid reason, selected optimization strategy), but delegated mechanism and final shape to us. Of the nine, model version and decision timestamp map to our decision log directly and entity ids map at candidate level; bid price, bid/no-bid, and no-bid reason belong to the Bid Decision/Response service; auction/request ids are the known serving-row gap; latency and optimization strategy come from the agent, not our rows. Assembling this dataset crosses lanes. Owned and designed by `09_monitoring.md` (a derived table plus an export job).

Provenance inventory (reference for the DE lane)

The batch-02 ML Service Background names the internal tables training data comes from, across two databases. The query engine is unstated in that report — Athena is our inference from the client's stack, not client-confirmed. These are the client's internal names; whether the curated datasets ship off them as-is or repackaged is the DE lane's question, not ours. Recorded as the provenance menu:

- `vir-prod-uaadsmanager-reporting-db` — auctions (`auction_us_east` / `auction_us_west`), bid logs (`bid_logs_us_east` / `bid_logs_us_west`), and a streamed win-log table (`silver_win_logs_stream`). Regional east/west splits mean the curation step unions them.
- `ua-data-jobs-prod-reporting` — batched `win_logs` (the likely source of the impression-level dataset), pixel events (`silver_kochava_events`), per-window web attribution (`gold_kochava_seven_day_attribution`; the report notes one/fourteen/thirty-day variants are available — the 1-day one is not in the client's data models and needs confirming), app events (`bronze_mobile_app_events`), and per-window app attribution (`gold_app_events_seven_day_attribution`, same window variants).

Two notes that cross back into our scope: the per-window gold tables match the union structure `02_datasets.md` gives `cleaned.attribution_labels`; and auction/bid tables appearing in a training-data list does not change the label universe — they are feature and analysis sources only (`03_labels.md`).

The account boundary

Owned by the Data Engineering and infra lanes; kept here as reference because it bounds what we can assume about freshness and read paths.

The bidder, and this whole offline platform with it, most likely runs in its own AWS account, separate from the client's core account; one client doc says "most likely", another frames the boundary as Terraform ownership within a shared VPC (with the bidder's security group added to the client's RDS ingress), so the account topology itself is not final. Either way, the client remains the source of truth for campaign configuration, delivery eligibility, attribution logic, and governed warehouse outputs, and crossing the boundary needs an explicit mechanism. The client names the candidates, undecided among them: cross-account IAM roles, Lake Formation sharing, S3 replication or export buckets, event streams, APIs, or other approved transfer mechanisms.

This is the substance behind working assumption 4 in `PLAN.md` (the five curated datasets are a given): something will cross that boundary, but which mechanism, and therefore what we actually read against, is not yet chosen — by the DE lane.

Source format

The client's warehouse is Apache Iceberg tables over S3, queried through Athena, governed by Glue Catalog and Lake Formation — plus Postgres as the source of truth for config and entity state. Attribution outputs land in both Postgres and the Athena-accessible tables. Whatever curated dataset we receive most likely inherits the Iceberg-over-S3 shape, either the tables themselves or a flat export off them; which one depends on the mechanism chosen above. Config/entity data (the campaign hierarchy) is Postgres-origin on the client's side; whether any curated dataset is delivered out of Postgres is open, and we do not plan direct Postgres reads.

Read path into Databricks

Decision: Unity Catalog external tables over external locations, reading data where it lands rather than copying it in first, matching working assumption 1.

Two cases, depending on the boundary mechanism:

- Lake Formation sharing of the Iceberg tables themselves: we read them directly, Unity Catalog's Iceberg read support covers this without a conversion step.
- A flat S3 export or replication bucket instead: Unity Catalog external tables over that bucket cover it the same way.

Either way, the raw layer (`02_datasets.md`) is a typed view over data that stays where the client put it; we don't maintain a second copy.

Athena's role

The client docs hold two positions. One expects the bidder to consume curated outputs only (quoted above). The other prescribes direct access infrastructure for the bidder: an Athena workgroup, Glue reads, Lake Formation grants, and Postgres connectivity on port 5432. The client has not reconciled them; resolving the access pattern is the DE lane's work (below).

Our lean, pending that answer: curated-first. Pipelines read through Unity Catalog only; direct Athena, if granted, serves exploratory analysis at most, never a pipeline dependency. We don't stand up a parallel Athena path, and we don't plan direct Postgres reads.

Access control

Read-only, scoped per source: one credential (IAM role, Lake Formation grant, or storage credential, depending on the mechanism chosen) per external location, least privilege, no write access to any client bucket or table. Writes into the serving-shared state (artifact store, rollout config) are the deployment job's alone (`07_deployment_job.md`); our pipelines write only our own catalogs and Redis, never the client location.

Cadence

The client's stated cadence tiers for what crosses the boundary, not yet finalized:

- Immediate / event-driven — delivery eligibility changes. Pacing and ingestor lane, not ours.
- Frequent / near-real-time — delivery settings, bid config, targeting. Pacing and ingestor lane.
- Hourly — delivery metrics, spend, impressions, clicks, win rate, bid/win diagnostics.
- Daily — attributed conversions, finalized reporting, training labels, feature snapshots.
- Weekly / backfill — large historical training sets, attribution reprocessing, feature rebuilds.

Our two label-side inputs land at the daily tier (the client names attributed conversions, training labels, and feature snapshots there) with large historical sets at the weekly tier. The tension that matters now: `04_features.md` proposed a 10-minute hot-feature job cadence against 30-minute windows, but the client's stated tier for delivery metrics, impressions, and clicks (the events hot features aggregate) is hourly. The client ingests raw delivery data near real-time on its own side (a dedicated Kinesis stream from the DSP), and its internal inventory includes a streamed win-log table (`silver_win_logs_stream`, provenance inventory above), so fresher boundary feeds are physically possible — but nothing fresher than hourly is currently stated for what crosses to us. Whether the streamed table can back a fresher curated feed is a question for the DE lane. Unless a fresher feed is negotiated, hot features work within an hourly freshness floor, and the 10-minute job cadence only reduces our own share of the delay. The "still to be determined" cadence language in the client's docs refers to the delivery-control feed, which is the pacing lane's input, not ours. The freshness budget is tracked as `04_features.md`'s open question 1.

Open questions

1. Which cross-account mechanism the client chooses (Lake Formation sharing, S3 export or replication, event stream, or API), and therefore the concrete read path and freshness we actually get. Owned by the DE lane; everything above adapts to whichever answer arrives.
2. Whether the five proposed datasets ship as named or get reshaped; exact schemas and bucket or table names either way. Owned by the DE lane, with our requirements as input (label universe and join key, per-window attribution structure and `event_name`, filled id triplets, freshness floor).
3. The boundary feed's real freshness for delivery events, which sets the hot-feature floor; the budget itself is owned by `04_features.md`'s open question 1, resolved together with question 1 here.
4. The bidder response dataset's assembly is designed (`09_monitoring.md`); what remains open is the client's required field list, cadence, and consumers, plus the Bid Decision/Response service's fields — their call and the agent lane's, respectively.

02 — Datasets

The storage layout on our side: what tables exist, where they live in Unity Catalog, how they are named, and the two conventions `00_overview.md` defers here (identity-space naming, privacy treatment). Exact inbound column schemas are open until the client's dataset inventory arrives (`01_data_access.md`); this doc fixes grains, keys, and rules, not column lists.

The layers

Each layer has one rule that defines it:

- Raw — Unity Catalog external tables reading the client's curated datasets in place; no copy, no transformation, the client's schema as shipped. Our rules do not apply here: the data is the client's, governed by their grants.
- Cleaned — the first Delta tables we materialize: typed, deduplicated, identity resolved (three ID spaces as separate columns), cost and spend units normalized (raw log values arrive in micros, reporting values in currency; the cleaned column states its unit), privacy rules applied. Reshaped the client content, nothing computed. Everything downstream of the client-origin data reads cleaned, never raw.
- Decisions — ingested serving events (the lib's decision-log rows), parallel to cleaned but with a different origin: our own serving side, not the client. Ingestion owned by `09_monitoring.md`.
- Derived — tables whose content we compute: the labels table (`03_labels.md`), the materialized feature tables (`04_features.md`), the per-arm readout tables and the monitoring tables (`09_monitoring.md`). Rewritable by partition until their inputs are final, unlike training snapshots.
- Training — point-in-time joins of derived labels, cleaned data, and feature values into model-ready training sets. Immutable snapshots, one per training run.

Catalog and naming

One catalog per environment (`ml_dev`, `ml_prod`) is our working assumption (`00_overview.md`), open until the account and MWAA questions land. The schema and table names below are our working proposal: the other docs use them as the working set, and they adjust on review, not on anything external.

Unity Catalog schemas map to layers plus one for decision-log tables:

- `raw` — external tables, one per curated dataset: `raw.impressions`, `raw.attribution_labels`, `raw.performance_hourly`.
- `cleaned` — same table names, materialized and conformed: `cleaned.impressions`, `cleaned.attribution_labels`, `cleaned.performance_hourly`.
- `derived` — computed tables: `derived.labels` (`03_labels.md`); the materialized feature tables `derived.feature_<model>_*`, one per model and join-key grain (`04_features.md`); the per-arm readout tables `derived.readout_<model>`; and the monitoring tables `derived.model_metrics` (one row per model, metric, cut, and day)

and `derived.data_quality` (one row per check, dataset, and day) — contents in `09_monitoring.md`.
- `decisions` — ingested decision-log tables, starting with `decisions.scored_candidates`. Populated by `09_monitoring.md`'s decision-ingest pipeline; the tables are defined here because they are dataset-layer objects like any other.
- `training` — one table per model, `training.<model_name>`, partitioned by `run_id`, which IS the MLflow run id of the training run that produced each row (one id, no mapping table). Partitioning is what makes the single table work: fetching one run's snapshot prunes to that run's files regardless of how many runs the table holds, and retention is a partition drop. Rows for a run are written once and never updated; Delta time travel is not the snapshot mechanism (vacuum erases history), the explicit `run_id` partition is.

Full name example: `ml_prod.cleaned.impressions`.

Grain and keys per dataset

Grains and primary keys are settled by what each dataset is; columns inside each row await the inventory.

- Impressions — one row per served impression. Two working assumptions to confirm with the client, tracked in `03_labels.md`'s open question 3: the dataset is wins-only, and it carries a key the attribution rows reference (their tables use `impression_id`; the client's delivery-log inventory documents `auction_id` / `request_id`). This is the training and label universe.
- Attribution labels — one row per attributed conversion. The cleaned table unions the client's per-window, per-family source tables with `source_family` and `window` columns; its key is (`conversion_id`, `source_family`, `window`), since the client guarantees `conversion_id` unique only within each source table. Carries the measurement-source keys (`enhanced_pixel_id` web, `app_tracker_id` app) and joins to impressions on `impression_id` (required in the attribution tables, nullable in the gold query layer; which layer backs the curated feed is open). The conversion rows' own `ad_id` column is nullable on the web-attribution provider side; that never affects the label join (identity comes from the impression row), but any rollup joining conversions directly to ads or campaigns loses the null-`ad_id` rows. `03_labels.md` tracks that loss as a data-quality metric.
- Performance hourly — one row per ad × hour. The client's sources scope it to pacing, reporting validation, spend reconciliation, and feature use, not labeling; we use it for reconciliation checks only, impression-level attribution is the label source. Its spend fields arrive in normalized currency while win-log costs arrive in micros; the cleaned layer's unit normalization is what makes the reconciliation comparable.
- Scored candidates — one row per candidate the serving lib scored. Joins served impressions and labels by (`request_id`, `candidate_id`); supports per-arm aggregation by model, version, arm, and revision.
- Labels (`derived.labels`) — one row per impression and label configuration; grain and columns in `03_labels.md`.

- Feature tables (`derived.feature_<model>_*`) — one row per join-key combination and time bucket; one table per model and join-key grain, features as columns, maintained incrementally by the materialization pipeline (`04_features.md`). A bucket is computed from cleaned partitions; when a cleaned partition is rewritten (late or corrected data), the buckets computed from it are recomputed. Nothing else reopens a bucket.

Identity-space naming convention

The rule `00_overview.md` assigns here. Three ID spaces exist (client UUID, DSP ID, raw-log ID); a bare id column is ambiguous and therefore banned from our layers:

- In `cleaned` and `training`, every entity reference is a column triplet with explicit suffixes, named by the client entity: `adset_client_id` (UUID, string), `adset_dsp_id`, `adset_raw_id` (the client's adset is the downstream "line item"; using the DSP-space name as the entity name would defeat the convention). Same pattern for campaign, creative, account, ad. A bare column like `adset_id` or `line_item_id` must not exist in these layers. Serving-side field names are a different world: a bid request's `line_item_id` is a serving field in the DSP/raw space, not one of our columns, and key templates keep serving names (`04_features.md`).
- We never create or compute these ids. The client controls identity mapping (stated), and their dataset guidance says all three ids should ship as separate, clearly named fields; whether every curated dataset actually will is unconfirmed, so the operative rule is the check: the cleaned layer renames the columns to this convention, verifies all three arrive filled, and reports any gap to the client rather than filling it ourselves. Every table built from cleaned keeps the same column names unchanged.
- `raw` keeps the client's names untouched; the convention applies from cleaned onward.

Privacy treatment

The rule `00_overview.md` assigns here. Applies at the raw → cleaned boundary; raw is the client-governed, cleaned onward is ours. Proposal: the final field-by-field list needs the schema inventory and the privacy compliance review (`PLAN.md` open question 4).

- Drop at cleaned — all IP-type fields (`ip_address`, `ipv6_address`, `impression_ip_address`, `event_ip`), all tracking and integration URLs (`postback_url`, `impression_tracking_url`), `user_email`, `event_user_agent`. Nothing we plan needs them; dropping beats governing.
- Hash at cleaned — `user_id`, `ifa` / `idfa`, device ids, `hhid`, `ga_client_id`: deterministic salted hash, same salt across datasets so joins survive, original values never stored. `user_id` arrives non-null on the web attribution tables (the client marks it required in that model; the app tables do not carry it), so it reaches us whether or not we use it; no documented join needs it, hashing is containment.
- Retain as-is — the consent signals (`is_gdpr`, `is_gdpr_consented`, `gdpr_consent_string`, `us_privacy`, `dnt`, `lmt`). They exist to gate feature use and retention; hashing them would defeat their purpose. One of them needs its own

review: `gdpr_consent_string` is a full TCF consent payload, and whether retaining it as-is is acceptable is a question for the privacy review, not settled here.
- No privacy-sensitive field, hashed or not, may appear in a `training` table unless a feature definition explicitly declares it (`04_features.md`); the training layer defaults to entity ids, features, and the label.

Retention and backfill

- Raw — no retention policy of ours; the data stays where the client put it under the client's lifecycle.
- Cleaned — partitioned by event date. Retention proposal: 13 months (one year of seasonality plus margin), to be sized against real volumes before prod.
- Derived — labels and feature tables follow their inputs: rewritable until final, retained on the cleaned layer's horizon.
- Training — snapshots retained while their model version is alive in the registry plus the rollback horizon; after that, deletable, since any snapshot can be rebuilt from cleaned.
- Decisions — retention driven by replay and readout needs; sized in `09_monitoring.md`.

Incremental vs. full rebuild

- Cleaned tables build incrementally by event-date partition, with idempotent partition overwrite: re-running a day replaces that day, so reruns and backfills are the same operation scoped to different partitions. No special backfill path.
- Attribution labels arrive late, so old partitions get rewritten. Label partitions are keyed by the impression's date, but a conversion arriving today can belong to an impression up to 30 days old (the longest window); writing today's arrivals therefore rewrites old partitions. For normal late arrivals, a partition stays rewritable until its date is older than 30 days plus the delivery margin; after that it is final in normal operation. A model using a shorter window sees its labels stabilize sooner (a 7-day label is stable after 7 days plus the margin); the 30-day bound is the blanket rule for the shared tables, not the wait every model pays. One event sits outside this rule: the client's attribution reprocessing runs at their weekly/backfill tier and may correct conversions in otherwise-final partitions; it is handled as an explicit backfill event that triggers downstream rebuilds, not absorbed by the margin.
- Training tables never rebuild in place: a new training run writes a new snapshot under a new `run_id`. Fixing a bad snapshot means a new run, not a patch.

Open questions

1. Exact inbound schemas, formats, and paths; carried from `01_data_access.md`, blocks finalizing raw/cleaned column lists.
2. Retention numbers (cleaned 13 months, decisions horizon) need real volume data before they are commitments.

3. The privacy compliance review, three parts: the final field-by-field drop/hash/retain list; whether deterministic hashing of user-level identifiers suffices (and whether the review constrains where hashed fields may flow); and whether retaining `gdpr_consent_string` as-is is acceptable.

03 — Labels

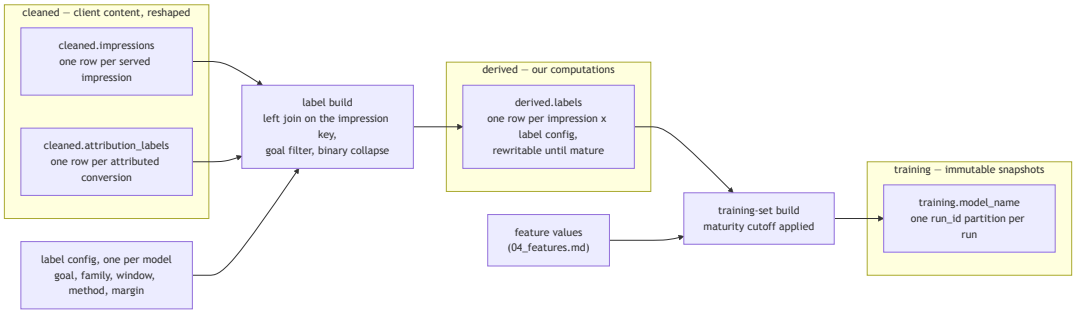
How the training label is built: which impressions count, which conversions count, how the two join, and when a label is final. Attribution itself is the client's pipeline's work; we consume its outputs and never recompute them.

The label

Per goal: an impression gets label 1 if the attribution data links at least one conversion of that goal to it within the attribution window, label 0 otherwise. The two MVP goals are Page View and Purchase, both initial priorities per the client; each gets its own label configurations. The universe is `cleaned.impressions`, under our working assumption, to confirm with the client, that this dataset contains wins only — every row a served impression, a scored-but-lost candidate never in it. The client describes the dataset as "joined delivery, campaign, ad set, ad, creative, supply, bid, win, spend, and event outcome data" without stating it is wins-only (open question 3).

The universe is served impressions by definition: `P(Action | Impression)` conditions on an impression being shown. Auction and bid data (which the client lists among training sources) can never be the label universe — a lost auction has no impression to condition on — and neither can our decision logs; both enter only as feature and analysis sources.

Dataflow



The label build runs on a schedule, keeps `derived.labels` current as conversions mature, and is independent of training; the training-set build reads whatever is mature when a run starts.

Inputs

- `cleaned.impressions` — one row per served impression. Which key it carries is unconfirmed: the client's attribution tables reference an `impression_id`, but the client's delivery-log inventory documents `auction_id` and `request_id` instead, and names `auction_id` as the bridge from conversions to delivery "where available". We

assume the curated dataset carries a key the attribution rows reference; which key is open question 3.
- `cleaned.attribution_labels` — one row per attributed conversion, from two source families: web (the web-attribution provider, conversions measured through an enhanced pixel: Pixel Purchase, Page View) and app (MMP trackers: App Install, out of MVP scope). Both families carry a required `impression_id` (the web table documents it as the DSP impression identifier; the app table does not name its space). Field names here come from the client's internal models; they are confirmed only when the curated feed's schema arrives.

Label parameters

One label configuration per model, registered with the model version in the registry (mechanics in `05_training.md`):

- goal event — which conversions count. Goals arrive as the ad set's `optimization_goal` values (`PIXEL_PURCHASE`, `PIXEL_PAGE_VIEW`; app events later); the filter column is `event_name`, and the mapping from goal value to `event_name` values needs the client confirmation (open question 1).
- window — 7, 14, or 30 days per the client's data models, and the client's ML report states a 1-day variant is also available (to confirm, open question 1). The client ships one source table per window; `cleaned.attribution_labels` unions them with a `window` column (`02_datasets.md`), so picking the window means filtering on it. The window may become advertiser-facing configuration (a goal-suffixed value like `PIXEL_PURCHASE_7_day`, or an ad set column), making it a product dimension: each goal uses a single window for now, and the expected end state is one label configuration and one model per (goal, window), each model trained on its own window's labels. Training on a shorter window and calibrating up to a longer one is a conditional alternative owned by `05_training.md`; the label build supports both unchanged.
- source family — web or app.
- method — click- versus view-through. The window tables carry no method field (a nullable `attribution_method` exists only on a legacy raw table), so today the parameter has one value: "as attributed by the client".
- maturity margin — extra days added to the window before a label is considered final, covering the proposed daily cadence of the attributed-conversions feed (the client's cadence expectation, not a live feed yet). Proposal: 2 days. Attribution reprocessing is handled separately (see Maturity), not by this margin.

Both goals ship in the MVP; what stays open is each goal's windows. Which windows the client will offer per goal is a product question (open question 1), and label volumes per (goal, window) decide what is trainable. Lean, not decided: per goal, the shortest offered window that yields enough positives.

Join mechanics

- Left join: every impression row survives; conversions attach by `impression_id`, filtered to the configured goal event.
- Conversions are scoped by their measurement source as well as the goal event: `enhanced_pixel_id` on the web side, `app_tracker_id` on the app side. The same `event_name` can arrive from different pixels; the source key keeps goals from colliding.
- The label is binary. An impression with three matching conversions gets the same label 1 as one with a single conversion. Conversion count and value (`price` on web, `revenue` on app) are kept as metadata columns for future value models; the MVP label ignores them.
- Conversion timestamps use the UTC fields (`event_timestamp_utc`; the web table marks it preferred for cross-system joins, the app table carries only the UTC form).
- A conversion whose `impression_id` matches no impression row is an orphan. Orphans are counted and reported per day, never silently dropped: a rising orphan rate means the impressions feed and the attribution feed disagree about what was served.
- The conversion rows' `ad_id` is nullable on the web side. This never affects the label (identity comes from the impression row); it only shrinks rollups that join conversions directly to ads or campaigns. The share of null-`ad_id` conversions is reported alongside the orphan rate.

Maturity

A label is final only once the window has closed and the data has arrived: impression time + window + maturity margin. Before that, the impression is immature — its label can still flip from 0 to 1.

Training uses matured impressions only. The training-set build drops every impression younger than (window + margin) at build time. This is the simplest rule that is correct, and it is the MVP rule: no weighting, no provisional labels. Training on younger impressions with early labels is a possible later iteration; the client's own docs anticipate it (early feedback datasets next to finalized attribution snapshots).

The cutoff guarantees the config's label is final for every row read: the window has passed, so normal late arrivals cannot flip it. Rebuilding a training set over the same date range therefore reproduces the same labels, with one exception: the client's attribution reprocessing (a weekly/backfill-tier event in their cadence plan) may correct conversions after the fact, including in otherwise-final partitions. Exact reproduction of a specific past run never depends on this; that is the training snapshot's job (`02_datasets.md`).

Output

`derived.labels` — one row per impression and label configuration (the config id subsumes goal, window, and family): the binary label, first-conversion timestamp, conversion count, conversion value, maturity status, and the label-configuration id.

Partitioned by impression date like its inputs, and rewritable on the same horizon (a late conversion flips a row from 0 to 1 until the partition is final).

The training-set build (`05_training.md`, with features from `04_features.md`) reads `derived.labels`. If it downsamples negatives, the keep fraction is recorded with the run; the base-rate correction that number feeds belongs to training and calibration (`05_training.md`), not here.

Open questions

1. Which windows the client will offer per goal (the client suggests advertiser-selectable windows), label volumes per (goal, window), whether the 1-day attribution variant named in the client's ML report exists as a table, and the mapping from `optimization_goal` values to `event_name` values. Blocks the window choice per goal; everything else here is parametrized and does not wait for it.
2. Whether the client's attribution exposes click- versus view-through on the window tables at all (only a legacy raw table carries a nullable `attribution_method` today).
3. The join, end to end: whether the curated impressions dataset is wins-only; which key it carries (`impression_id` versus `auction_id` / `request_id`); and which layer the attribution feed derives from (attribution tables where `impression_id` is required, or the gold query layer where it is nullable — and where `event_name`, the goal filter, is also nullable and ids arrive string-typed). The label universe and label coverage both depend on it.
4. The maturity margin value (proposal: 2 days), and how attribution reprocessing events are detected so downstream rebuilds trigger.

04 — Feature layer and hot-feature jobs

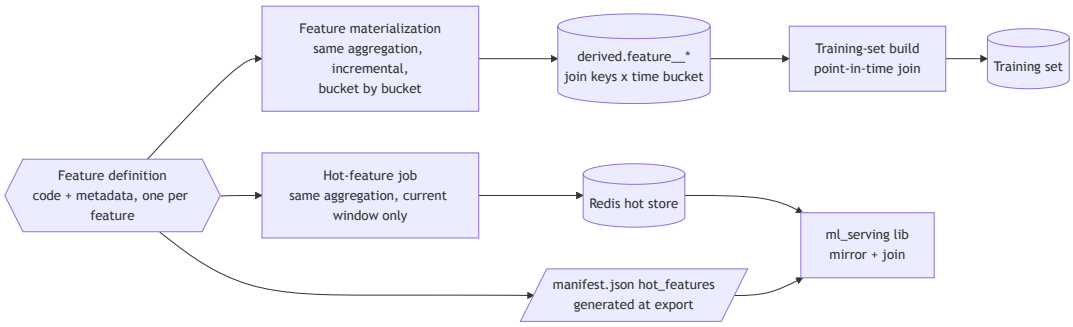
The shared feature layer: one definition per feature, consumed by both the training-set build and the batch hot-feature jobs. Owns everything about hot features that the serving lib deliberately does not: windows, aggregation, key rendering, freshness, and lifecycle.

The no-skew contract

Every feature has exactly one definition: one aggregation (for example, clicks counted and grouped by publisher, over a 30-minute window), computed by one piece of code. That code runs in two places:

- the feature-materialization pipeline runs it incrementally across history, bucket by bucket, into the derived feature tables (Feature materialization, below); the training-set build joins those tables point-in-time, so each training row gets the value as it was at its own moment. Running the definition inline over a run's range is the fallback when a definition version is not materialized yet.
- the hot-feature job runs it once, over the current window, to populate Redis.

The grouping, the metric, and the window length are identical in both places; only how many times the code runs, and over what span, differs. The value a model trained with and the value the serving-time join finds must come from that one definition; divergence between the two is train/serve skew, the failure mode this whole layer exists to prevent. The contract is proven at dev scale in the `ml_serving` repo: one Python function feeds both the training join and the Redis seeder, and the exported artifact gates on an ONNX-versus-Python parity check at 1e-4, re-verified end to end by the repo's smoke test at the same tolerance.



What a definition declares

A feature definition is Python code plus declarative metadata, both in the shared layer:

- name — also the ONNX input name and the manifest `hot_features.name`.
- input events — which Delta tables feed the aggregation.
- aggregation and window — the computation and its lookback (for example, impressions per line item over the last 30 minutes).

- join keys — field names, composite allowed. The definition metadata declares each key's ID space (client UUID, DSP ID, or raw-log ID, per the identity-mapping rule in `00_overview.md`); a join-key declaration without an ID space is invalid. The key string itself (and the `key_template`) stays plain.
- key_template — the Redis key name for one dimension-combination's value, with `{field}` placeholders filled in per combination (a feature grouped by publisher needs one key per publisher, not one key for the whole feature). Starts with a namespace prefix owned by the model whose feature set declares the definition (the anti-collision contract), for example `pixel_purchase:li_clicks_30m:{line_item_id}`.
- default — a float used when the key is absent. The default means cold start (a line item with no history yet), not error masking; it must be a value the model can meaningfully consume.
- privacy class — whether the feature reads any privacy-sensitive field (`02_datasets.md`'s treatment rules). Required declaration; it is the enforcement point for the rule that no sensitive field enters a training table undeclared.

The manifest's `hot_features` section is generated from these definitions at export time, never written by hand. One source of truth: if training, Redis, and the manifest disagree, the bug is in exactly one place.

The two consumers

Example: `clicks_per_publisher_past_30min`, clicks counted and grouped by publisher, over a 30-minute window.

For training, each training row needs the feature's value as it was at its own moment, not today's value. The materialization pipeline buckets history into successive 30-minute windows and runs the aggregation once per bucket, per publisher, maintaining a time series of values in the feature's derived table; each training row joins to the bucket covering its own timestamp. The time series is extended incrementally as new cleaned data lands, never recomputed per training run.

For the hot-feature job, only one number per publisher is needed, right now. It runs the same aggregation, clicks grouped by publisher, over a single window: the past 30 minutes, ending now. No buckets, no history, one value per publisher, written to Redis under that publisher's rendered key_template.

Same metric, same grouping key, same window length. The only difference is that training evaluates it repeatedly across a historical range (one value per bucket) and the hot-feature job evaluates it once, for the current moment. A bucket or key with no history gets the declared default, the same value in both places, so the model learns what the default means.

Key rendering must match the Go assembler's rules: strings pass through as-is; numeric key values format as the shortest round-tripping decimal in fixed-point form, never exponent notation (an id of 4521.0 renders "4521"; Go's `strconv.FormatFloat(v, 'f', -1, 64)` semantics). The fixed-point constraint is the trap: Python's default float

formatting switches to exponent form at large and small magnitudes and would silently render a different key. The shared layer provides the one rendering function both jobs and any validation use.

Feature materialization

The offline half of the feature store: a batch pipeline, ours, that executes the definitions over the cleaned tables and maintains the historical aggregates as Delta tables, so training runs join instead of recompute.

- Tables — `derived.feature_<model>_*`, one table per model and grain (one set of join keys, one bucket length). A model's features sharing both ride in the same table as columns; a feature with the same keys but a different window stepping gets its own table. Example: model A with `metricA` and `metricB` on keys (X, Y) and `metricC` on key Z produces two tables — `derived.feature_modelA_x_y` with rows keyed (X, Y, bucket) and columns `metricA` / `metricB`, and `derived.feature_modelA_z` with rows keyed (Z, bucket) and column `metricC`. The same three metrics appear in Redis as flat per-key floats (Keyspace management, below); the table grouping exists only offline. Ownership is per model by design: a formula two models both declare is computed per model, under each model's tables and prefixes — ownership over deduplication. One point-in-time join per grain at training-set-build time, however many features ride on it.
- Grain — join keys × time bucket; the bucket length is the definition's window stepping (a 30-minute-window feature produces 30-minute buckets).
- Incremental rule — buckets are computed as cleaned partitions land and partitioned by bucket date; a late or corrected cleaned partition recomputes the buckets it feeds. A bucket stays rewritable for as long as its cleaned inputs are, then it is final with them (`02_datasets.md`). Each day costs one day of computation, not a full-lookback recompute per training run.
- Versioning — materialized values carry the definition version that computed them. The version identity is per definition — a content hash of that definition's code and metadata (or an explicit version field), so it changes exactly when the definition changes; the repo commit that produced the rows is recorded alongside as lineage, but never used as the version key, since unrelated commits would mark every value stale. A changed definition is a new version and needs its backfill before training runs can join it; until the backfill lands, a run whose feature set names the new version computes that feature inline over its own range (the fallback). The training-set build checks the model configuration's feature-set versions against the tables and chooses join or fallback per feature.
- Scheduling and ownership — scheduled alongside the hot-feature jobs, same definitions, same shared layer; the DAG repo schedules, the workload repo computes.
- Hot-feature job — unchanged by materialization: it computes the current trailing window straight to Redis. The materialized tables serve history; Redis serves now.

Implementation lean, not decided: Unity Catalog feature tables with point-in-time lookup are the Databricks-native fit for exactly this shape.

Repo home

The shared layer is part of the workload package. Both consumers are Databricks workloads, so both import it inside their runs; the package ships as versioned wheels, pinned per pipeline (`08_orchestration_repos.md`). The DAG repo defines and schedules pipelines but never contains feature logic; the only workload code it imports is the config-schema subpackage, in CI.

Freshness and cadence

Three delays stack in series: how fresh the inbound curated data is when it reaches us, our job cadence, and the lib's whole-keyspace mirror interval (5–10 minutes; serving reads a snapshot, not Redis). Worst-case staleness at scoring time is the sum.

The first delay is the binding one and is not ours to set: the client's stated cadence tier for delivery metrics, impressions, and clicks (the events hot features aggregate) is hourly, so unless a fresher boundary feed is negotiated, hot features carry an hourly freshness floor regardless of how often our jobs run (`01_data_access.md`).

Our sizing proposal for the part we do control: one hot-feature pipeline, parametrized over the models' feature sets (`08_orchestration_repos.md`), runs every 10 minutes as an MWAA-triggered Databricks run, adding at most 20 minutes (10 of job cadence plus 10 of mirror interval) on top of the inbound delay. Cadence is a per-feature-set parameter, not a platform constant; a feature whose window is hours can run hourly.

Keyspace management

- Shape — one key per metric per key-combination, value a single float for the current window; tables and buckets exist only offline. A model with `metricA` and `metricB` on keys (X, Y) and `metricC` on key Z produces keys like `modelA:metricA:x1:y1`, `modelA:metricB:x1:y1`, `modelA:metricC:z1` — the join-key values are in the key name, metrics never share a key.
- Namespace — every key starts with the owning model's prefix. Nothing writes outside its model's prefixes; collisions are a review-time check on key_templates.
- TTL — every key gets a TTL of 3x its job cadence (proposal). Keys for retired line items expire instead of living in the lib's in-memory mirror forever; a healthy job refreshes its keys long before they expire.
- Size budget — the lib mirrors the whole keyspace into every Bidding Agent process, so total key count is a real budget, not a Redis concern. Each definition estimates its key cardinality (bounded by its join-key space, for example active line items, not users); anything user-keyed would blow the mirror and is out of scope for this store.
- Write mechanics — idempotent pipelined SET with TTL, full overwrite per run. No deletes needed: TTL handles retirement. If a run fails, existing values go stale, then expire, and the lib falls back to declared defaults per key. This degrades prediction quality without failing requests. One dependency to flag: serving the default on a missing key is the lib's current v0 behavior, and whether defaulting should ever gate a

request instead is an open question on the serving side; this TTL design assumes the current behavior stands. Job failures must still alert (see `09_monitoring.md`).

Feature lifecycle

Ordering rules, both directions:

- Adding: add the definition, backfill its training values, deploy and verify its hot-feature job, and only then train and ship a model declaring it. A manifest must never declare a feature whose job is not live; the deployment job's validation step checks that the feature's keys exist in Redis before writing a config.
- Removing: ship a model that no longer declares the feature, wait out rollback horizons (the previous version may still be re-promoted), then stop the job and delete the definition. TTL clears the keys.

Planned feature families

Beyond request-derived fields (free at bid time, not this doc's concern), two aggregate families are planned; both flow through the same definition layer as any other feature:

- Advertiser-level rolling aggregates — conversion and delivery rates at advertiser, campaign, and ad set grain. These are the sparse-signal mechanism: one global model per goal personalizes through these features as an advertiser's history accumulates, so no per-advertiser model is ever needed. The client requires that the approach not depend on per-advertiser history; personalizing through features is our chosen mechanism. A new advertiser has no keys yet and gets the defaults — cold start is the declared-default case, not a special mode.
- Auction and bid-landscape aggregates — win rates, price distributions, auction volume by segment, sourced from the client's auction and bid data via the curated feeds (`01_data_access.md`). Candidate features only; they never define the label universe (`03_labels.md`).

Open questions

1. The freshness budget per feature: the inbound-data floor (hourly as currently stated, unless a fresher boundary feed is negotiated — the client's internal inventory includes a streamed win-log table, so fresher is physically possible; see `01_data_access.md`) plus what staleness each feature can tolerate. Needs real features and product input. Until then, 10-minute cadence and 3x TTL stand as proposals for our share of the delay.
2. Whether key-presence validation at deploy time checks existence only or also value distributions (a job writing zeros passes an existence check).
3. Real keyspace sizing. The flat key-per-dimension-combination shape (above) matches how online feature stores work everywhere (offline = table, scanned in bulk for training; online = key-value, point lookup per request); that split is not in question. What is not yet sized: total key count once real metrics and their join-key cardinality exist, since the lib mirrors the whole keyspace into every Bidding Agent process's

memory rather than doing a live lookup per request. Moderate cardinality (hundreds to low thousands per metric) is cheap to mirror; a metric with high-cardinality join keys (user-level, for example) would not be. Revisit once real metrics are known; options if any metric runs too large: a live per-request lookup for just that feature instead of mirroring it, or grouping several metrics sharing a join key into one Redis hash per combination instead of one flat key per metric (the latter needs a serving-lib read-path change, a cross-repo decision).

05 — Training

The training pipeline: from matured labels and feature definitions to a registered model version carrying a parity-checked serving artifact. Upstream, `09_monitoring.md` owns when runs happen and MWAA triggers them; downstream, registration in Unity Catalog starts the deployment job (`07_deployment_job.md`), which publishes what this pipeline already produced. This doc is the full design; which capabilities the first implementation activates is an operational decision, not part of it.

Model configuration

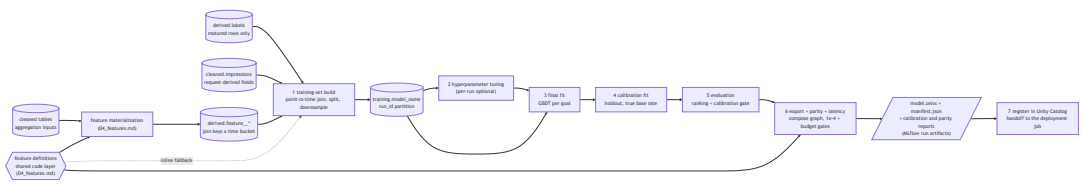
One configuration per model slot, versioned, and the single parametrization point of the pipeline:

- model name — the slot name, by goal (`pixel_purchase`, `pixel_page_view`); extends to goal and window (`pixel_purchase_7d`) when windows multiply.
- label configuration id — goal event, source family, window, method, maturity margin (`03_labels.md`).
- feature set — the feature definitions used, with their versions (`04_features.md`), plus the request-derived fields the model consumes, each declared with its source and dtype (what the export writes into the manifest's input schema).
- hyperparameters — either fixed values or a tuning spec (search space, budget, objective); see Hyperparameter tuning.
- calibration configuration — method (Platt or isotonic), advertiser layer settings (on/off, eligibility threshold, shrinkage strength), window variants to export.
- data ranges — training lookback, calibration and evaluation holdout lengths.
- downsampling policy — negative keep fraction and stratification, or none.
- rollout policy — a sibling definition in the pipeline folder (`08_orchestration_repos.md`), contents owned by `06_validation_promotion.md`; the training run does not consume it, it passes it through for registration.

The configuration and the rollout policy are registered with the model version the run produced, closing the lineage `03_labels.md` defers here: a version always knows which label configuration, feature versions, calibration settings, and rollout policy belong to it, and the deployment job reads the policy from the registry.

Pipeline structure

One pipeline definition, instantiated per model configuration, submitted as a Databricks job run on an MWAA trigger (`08_orchestration_repos.md`). The pipeline does not know why it was triggered; schedules and degradation triggers are `09_monitoring.md`'s.



Cylinders are tables, the hexagon is code (the shared feature-definition layer), rectangles are pipeline stages, the parallelogram is the run's output bundle. The feature materialization pipeline is `04_features.md`'s component, drawn here because it produces the build's feature input. The dotted edge is the fallback: when a feature's configured version has no materialized values yet (new feature, or a changed definition whose backfill hasn't run), the build executes that feature's code itself, over its own range only, instead of reading the table. The definitions feed the export stage too: the manifest is generated from them.

Run modes

Over the same stages:

- full train — all stages, fixed hyperparameters (stage 2 skipped).
- tune and train — stage 2 enabled by the tuning spec.
- recalibrate-only — skip stages 2–3: rebuild a fresh calibration slice, refit the calibrator over the existing trees, re-run evaluation, export, and registration. Cheaper than a retrain when the model ranks well but drifts in calibration. The result is a new version like any other: the calibrator never changes independently of a registered version, so model and calibrator stay a matched pair.
- window-variant export — stages 4–7 once per configured window: the same trees with a calibrator fit on that window's matured labels, one artifact and one registered version per window. This is the train-short-and-calibrate-up alternative to training each window's model separately; its validity gate is the ranking-invariance measurement in evaluation.

Training-set build

- Universe — the matured rows of `derived.labels` for the model's label configuration, over the configured lookback. Maturity is `03_labels.md`'s cutoff (window plus margin at build time), so every label read is final under normal operation.
- Features — request-derived fields come from the impression row itself (`cleaned.impressions`); aggregates come from the materialized feature tables (`derived.feature_<model>_*`, `04_features.md`) by point-in-time join, each row picking the bucket covering its own timestamp. A feature whose definition version is not materialized yet is computed inline over the run's range with the same shared code — the fallback, never the normal path.
- Split — temporal, three ways: train, calibration holdout, evaluation holdout, in time order. The calibration and evaluation holdouts keep the true label distribution; only the train split may downsample.
- Downsampling — negatives only, parametrized by the policy, keep fraction recorded with the run. Sampling is stratified by advertiser so the global model keeps seeing small advertisers instead of a uniform sample dominated by the largest ones; stratification keys and rates are part of the policy.
- Snapshot — written to `training.<model_name>` under the new run id (`02_datasets.md`), with a split column, so the snapshot holds the exact rows and the exact split of the run. Immutable once written.

Hyperparameter tuning

A pipeline stage, present in the design and enabled per run by the tuning spec in the model configuration:

- search space and budget (trials or wall time) come from the spec; the objective is a ranking metric on an internal temporal validation slice carved from the train split. The calibration and evaluation holdouts are never visible to tuning.
- trials run as nested MLflow runs under the pipeline's run, so the search is inspectable and the winner reproducible.
- the winning parameters become the final fit's parameters and are recorded in the configuration lineage as the resolved values.

A run without a tuning spec uses the configuration's fixed hyperparameters unchanged. Tuning cadence is an operational choice: nothing in the pipeline assumes it runs every time.

Model fit

One global gradient-boosted tree model per goal (LightGBM), trained on CPU, over all advertisers. Advertiser, campaign, and ad set identity plus their history aggregates enter as features; this is what serves new and low-signal advertisers without per-advertiser models — a new advertiser scores through the request and setup features until its aggregates accumulate.

Class imbalance is handled in the data, by the build's downsampling, not by loss weighting. Decision, not accident: a recorded keep fraction has a closed-form probability correction, while loss weights distort scores in a way no closed form undoes, and the output of this pipeline must end as a true probability.

Categorical fields are encoded inside the exported graph, so the training-time encoding is chosen for exportability: the encoder fit at training is the one baked at export, never a separate serving implementation.

Calibration fit

Two steps, in order, fit on the calibration holdout (true base rate):

1. Base-rate correction — closed-form, from the recorded keep fraction; skipped when the run did not downsample.
2. Calibrator — Platt or isotonic per the configuration. Lean: isotonic when the holdout has enough positives to support a stepwise fit, Platt when sparse; the threshold is set at EDA with real volumes.

The advertiser layer, when enabled in the configuration, fits on the same holdout as a grouped calibration: one global calibrator plus per-advertiser parameters for every advertiser above the eligibility threshold (minimum matured conversions in the holdout), each shrunk toward the global fit in proportion to its volume. Below the threshold an

advertiser simply has no row and gets the global behavior. The result exports as a lookup inside the graph — advertiser id to parameters, with the global fit as the default row — so serving stays a single self-contained artifact and a batch can mix advertisers freely.

Window variants reuse the fitted trees and fit one calibrator per configured window on that window's matured labels, producing one artifact per window (see Run modes).

Evaluation

A gate: ranking and calibration are checked against absolute thresholds, configured per model. Fail → the run stops here: nothing registers, nothing downstream runs, an alert fires.

On the evaluation holdout:

- ranking — AUC and PR-AUC (PR-AUC is the informative one at these base rates) against an absolute floor.
- calibration — reliability curve and expected calibration error, computed on the corrected, calibrated output, against an absolute floor.
- standard cuts — the same metrics by advertiser (per-advertiser calibration error is the signal that tells us when the advertiser layer earns activation), by campaign, and by time slice within the holdout.
- ranking invariance — when window variants are configured: whether the long-window conversion rate is a stable monotone function of the short-window score across segments. Stable supports calibrating up; unstable means that window needs its own trained model.

All metrics land in the MLflow run. Threshold values are set per model configuration.

A rollout policy's A/B stage (`06_validation_promotion.md`), when a policy enables one, re-introduces a champion-relative comparison — live, not offline replay. This gate and that one answer different questions: this one asks "is the model good," A/B asks "is it better, measured against what's currently live."

Export, parity, registration

- Export composes the self-contained ONNX graph: in-graph preprocessing (the encoders fit at training), the trees, and the calibration — including the advertiser lookup when that layer is active. One input tensor per raw field, named identically to the field; raw strings pass through; single output `p_cal`; the file named per the manifest's `onnx.file` (convention `model.onnx`).
- The manifest is assembled at export, never written by hand: the hot-feature join spec (join keys, `key_template`, default per feature) from the feature definitions; the input schema (name, source, dtype per field) from the model configuration's feature set, request-derived fields included; plus `onnx.file`, `output_name`, and `mlflow_run_id` as the lineage pointer, `model_name`, and `version`.
- Parity is a hard gate: the Python pipeline and the exported graph are scored on held-out rows and the run fails if max |diff| exceeds 1e-4. The serving repo applies the

same tolerance in its dev-scale smoke test (ml_serving).

- Inference latency is also a hard gate at this step, same moment as parity, different property (speed, not correctness): the exported graph is timed, and the run fails if it exceeds the configured budget.
- `model.onnx`, `manifest.json`, the calibration report, the parity report, and the monitoring references — the holdout score distribution and per-feature input distributions with null shares (`09_monitoring.md` reads them as drift baselines) — are logged as run artifacts; registering the version in Unity Catalog is the pipeline's last act and the deployment job's trigger. Registration triggers the deployment job through a link set on the registered model in Unity Catalog ("on a new version, run this job"); for a brand-new slot that link does not exist yet, so the pipeline sets it when it registers the slot's first version — once per model, a no-op afterwards. Automating it here means a new slot can never silently register versions that no rollout picks up. The job's export task publishes these files unchanged (`07_deployment_job.md`); nothing downstream rebuilds or re-checks them.

Lineage and reproducibility

The MLflow run records the resolved model configuration (label configuration id, feature versions, final hyperparameters, calibration settings), the rollout policy, the data ranges and split boundaries, the keep fraction, all evaluation metrics, and the parity result. The run id is also the snapshot partition in `training.<model_name>`, so the chain is one lookup deep in each direction: registered version → run → snapshot → configurations.

Exact reproduction of a past run reads its snapshot (`02_datasets.md`). Rebuilding over the same date range reproduces the same labels for the same configuration, with the client's attribution reprocessing as the one documented exception (`03_labels.md`).

Open questions

1. Label volumes per goal and window gate three knobs at once: the downsampling policy, the Platt-versus-isotonic threshold, and a sensible tuning budget. All set at EDA.
2. The advertiser layer's eligibility threshold and shrinkage strength, set against calibration-fit variance on real per-advertiser volumes.
3. Lookback and holdout lengths per goal, against volume and seasonality (a purchase model likely needs a longer lookback than a page-view model).
4. Categorical encoding for high-cardinality id fields (vocabulary with an out-of-vocabulary bucket versus hashing), decided with the first real feature set, since it must round-trip through the in-graph encoder.

06 — Validation and promotion

The decision logic of the deployment job. One job, two docs: `07_deployment_job.md` owns the mechanics (tasks, triggers, writes); this doc owns what the tasks decide on — the rollout policy, the per-state readout verdicts, the guardrails, and where humans approve.

Decision inputs

Everything a verdict reads comes from three places:

- the training run's own gate (`05_training.md`): ranking and calibration checked against absolute thresholds before the version ever registers. A version that fails never registers, so the job can assume quality is already settled.
- the per-arm readout tables (`09_monitoring.md` builds them): decision rows joined with matured labels, aggregated per model, version, arm, and revision. This is also where an enabled A/B's champion-versus- challenger comparison is read from.
- guardrail signals: spend, win rate, delivery, and the serving-side health metrics (latency, errors, defaulted-feature rates). Latency, errors, and defaulted-feature rate are agent-internal, continuous. Spend, win rate, and delivery depend on auction-outcome data the agent does not observe directly (open question 5) — the only confirmed path for win rate and delivery is the client's curated feed, hourly at best (`01_data_access.md`'s cadence tiers), not continuous.

The challenger, shadow, and routing capabilities this doc reads from are the serving repo's target design; its v0 ships champion-only. The doc designs against the target, consistent with the rest of the set.

Rollout policy

The decision logic is parametrized per model by a rollout policy — a sibling of the model configuration in the pipeline's definition folder, registered with the model version alongside it (`08_orchestration_repos.md`):

- thresholds — the readout bounds listed in the sections below; values are per model because base rates and volumes differ per goal.
- bake times — minimum time in each state before a verdict, and the maximum hold (below).
- canary ramp — shape and the traffic level that counts as the promotion point.
- gate 1 — on or off. Off (the default) auto-proceeds once shadow's bake time is green; on holds for a human before the first live bid.
- A/B — on or off. Off is the common case (see Retraining modes); on adds the A/B test definition below to the ramp.
- evidence mode — what promotion waits for: `full_window` or `proxy` (see Evidence modes). Per model and per change: a routine refresh of an incumbent can reasonably promote on proxy evidence, a new model or a changed feature set should wait for

the full window. Lean, not a rule: recalibrate-only runs default to `proxy`, first versions of a model default to `full_window`.

Gate 2 (promote) is fixed for every policy and is not part of the policy (see Approval gates). That is, no model goes to PRD unattended.

Retraining modes

Three things trigger a new version; each maps to a rollout policy. Training mode and rollout policy are independent — the mapping below is what a human picks, not a rule the system infers from how a version was trained.

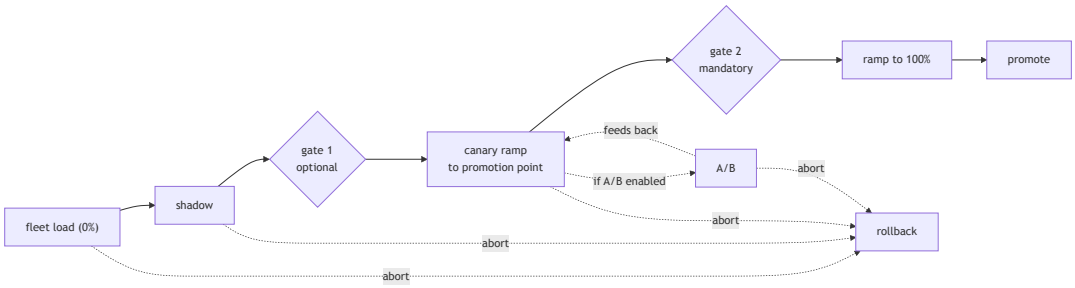
Trigger	What's different about the model	Rollout policy
Scheduled	Same architecture/features/HP — just a fresher training window	Gate 1 off, A/B off
Alert (drift/calibration monitoring)	Usually nothing architectural — reacting to a detected problem	Same policy as scheduled, unchanged
Engineer/experiment	Whatever the engineer changed — new feature, architecture, anything judged higher-risk	Gate 1 as chosen, A/B on

Success criteria — what "good enough to ship" means per mode:

Mode	To become champion, must…
Scheduled	Clear training's absolute gate (ranking, calibration, latency) → shadow green (no distribution anomalies) → canary ramp green (guardrails: spend, win-rate, latency, errors) through the promotion point
Alert-triggered	Same bar as scheduled. The gate is absolute, not champion-relative — it doesn't matter that champion is the thing that drifted
Experiment	Same training gate, plus a clean A/B: guardrails green for the full bake, calibration/ranking within the A/B's own bounds, non-inferior to champion (trusted only if champion itself is healthy). "Genuinely better" is never a gate condition — read from the A/B numbers afterward, not required to ship

Rollout states and readout

The advance and readout tasks loop through the states; each pass produces one verdict: continue, hold, or abort.



- Fleet load (0% traffic) — the artifact loads fleet-wide without scoring anyone. Reads what is observable at 0%: the lib's reload-error counters, load failures the agent surfaces at startup, and fleet health (memory, restarts) from the serving-side monitoring. Continue when the fleet is healthy for the bake time.
- Shadow — the challenger scores real traffic out-of-band; no bid is affected. Reads, no labels needed: the challenger's score distribution against the champion's on the same traffic, defaulted-feature rates (a spike means a declared field is missing — a hot-feature job lagging the manifest, or the agent not sending a field), latency contribution, and sampling integrity. Continue when distributions are stable and health is green for the bake time.
- Canary ramp — live bids, ramping up to the policy's promotion point. Checks: delivery guardrails (spend, win rate vs. the champion arm), realized calibration on whatever window has matured labels, and a sample-ratio check.
- A/B (only if the policy turns it on) — champion vs. challenger at a configured traffic split, nested inside the canary ramp. Checks per arm: realized calibration, cost per action vs. the goal, spend, and delivery.

Hold means the bake extends because the evidence is inconclusive — not enough matured labels or volume yet — and is bounded: past the maximum hold the run escalates to the humans instead of waiting forever. Integrity rules apply throughout: the arms differ only in the model (same features, same pacing), the routing salt stays fixed for the experiment's lifetime, and any sample-ratio mismatch voids the state's readout.

A/B test definition

Only relevant when a policy's `ab_test` is on (Retraining modes: the experiment case).

Parameter	Definition
Metrics	ranking (absolute floor + margin vs. champion), calibration (absolute floor), cost-per-action vs. the goal — per arm
Traffic split	configurable ratio + split key. Split key default: per-request (largest, fastest-accumulating sample). Coarser keys (per-advertiser, per-line-item) trade sample speed for exposure consistency — a deliberate choice per test, not a default
Arm size	statistical power calculation — needs a baseline conversion rate and a minimum detectable effect (open question 3)
Duration	derived, not set directly: evidence mode × arm size × actual traffic volume. `full_window` waits for the real attribution window to mature; `proxy` promotes on the shortest available window plus guardrails

Concurrency during an active A/B

Only one challenger per model slot, same invariant as always. While an experiment's A/B is in flight, promotion is frozen for that slot — champion has to stay fixed for the comparison to mean anything. Scheduled and alert-triggered retrains keep happening underneath and queue, the latest superseding whoever's waiting once the freeze lifts. Mechanics (the `queued` field, registration-time slot check, re-trigger on resolution): `07_deployment_job.md`.

Guardrails and automated abort

Always on, every state, including after promotion — a champion is guarded for its whole life, not only its own rollout. No human needed to trigger an abort.

Signal	Fires as fast as
Serving error or latency breach	continuous — the agent observes it directly
Defaulted-feature-rate spike	continuous — the agent observes it directly
Spend anomaly (challenger arm)	the underlying data arrives (open question 5)
Win-rate collapse	the underlying data arrives (open question 5)
Calibration blowout on early labels (realized conversion rate off the prediction by more than a configured multiple)	labels mature

Any breach aborts: the promote task's rollback branch writes a config pointing at the previous version (`07_deployment_job.md`). Abort is cheap and reversible — a rolled-back version can re-enter the loop after a fix — so guardrails are tuned to fire early.

Two manual controls exist alongside the automated path, available to a human independent of any breach:

- **Red button** — trigger the abort path manually, at any point, for anything a threshold wouldn't catch.
- **Rollback button** — move champion → previous on demand, any time after promotion, without needing a guardrail breach as the trigger.

Approval gates

Gate 2 (canary → promote) is fixed and mandatory for every policy — before the version becomes champion. This is the one invariant that never turns off: no version reaches the champion slot unattended.

Gate 1 (shadow → canary, before the first live bid) is a policy parameter, off by default (Retraining modes). When on, it holds the same way gate 2 does.

Every other transition advances automatically when its state's metrics are green for the bake time. Aborts are always automated; a guardrail never waits for approval.

Evidence modes

The label windows set the clock: a 7-day goal's outcomes are not measurable for a week, a 30-day goal's for a month. The evidence mode in the rollout policy decides how promotion handles that:

- `full_window` — each labeled verdict waits until the goal's own window plus margin has matured for enough arm traffic. The strongest evidence and the slowest path; for a 30-day goal, impractically slow for routine refreshes.
- `proxy` — verdicts read the shortest offered window's labels (the 1-day variant the client's ML report names — absent from the batch-01 data models, so confirming it is part of the windows question — or else the shortest that exists) plus the guardrails, and promotion proceeds on that fast evidence. The full-window readout does not stop at promotion: the promote task stamps a `trailing_until` tag on the model version, and the deployment job's own scheduled passes keep judging it against the full window's maturing labels until the tag expires — rollback armed if the matured picture contradicts the fast one (`07_deployment_job.md`).

The proxy mode's validity is not free: it rests on the long-window outcome being predictable from the short-window one, which is the same ranking-invariance evidence the training evaluation measures. A model or goal that fails that check must use `full_window`, whatever the policy says — the gate enforces it.

Launch readiness

A model version is ready to serve when, in order:

1. its training run passed the export parity, latency, ranking, and calibration gates
2. it ran in shadow on live traffic with no distribution or health anomalies
3. its realized calibration on live, won impressions stayed within tolerance over the configured evidence window

4. the delivery guardrails stayed green through the canary ramp, including A/B when the policy runs one
5. a human approved the final promotion — and the first live exposure too, when gate 1 is on

Calibration is evaluated on won impressions: the readouts join served impressions to their outcomes, the same population cost per action is computed over, since spend only happens on wins.

Open questions

1. Threshold values and bake times: need label volumes per goal and window, and the client's observed baseline conversion rates. Set at EDA; the structure above does not wait for them.
2. The cost-per-action evaluation window and the treatment of fees — the client's ML report marks both as remaining to be defined, without assigning an owner.
3. Minimum arm sizes for a powered A/B per goal: needs volume data; until then A/B bake times are conservative.
4. Which role holds the two approvals — a team decision, recorded when the job goes live.
5. Spend and win-rate data provenance for the guardrails above: whether the ad server gives the agent a real-time win-notice/price signal (the client's own non-functional requirements list win/loss and spend/pacing as agent-monitorable, alongside clearly-synchronous metrics, which hints one exists) or whether the agent only ever learns win/spend outcomes through the hourly curated feed. The client can answer; until then, the spend and win-rate guardrails above are designed but their actual latency floor is unknown.
6. A/B cost and duration are unknown — they depend entirely on how fast labels mature. If maturity is a matter of hours, A/B is cheap and viable often; if it needs the full attribution window, it's expensive and should stay reserved for genuinely high-risk changes. Don't assume either regime until EDA says which one we're in.

07 — Deployment job

The deployment job itself: what triggers it, its task structure, and the exact writes it makes into the two touchpoints serving reads (the artifact store and the rollout config). One job, two docs: `06_validation_promotion.md` owns the decision logic each task runs on (gates, thresholds, approval placement); this doc is the mechanics only.

Trigger

A new version registering in the Unity Catalog model registry starts one run of the job, attached to that registered model (in Databricks terms, a Lakeflow deployment job defined in Asset Bundles). The attachment and that shape are our design choices; the serving repo's adopted docs establish the job's responsibilities (evaluate, export, advance, read out, promote or roll back) and that only it writes the touchpoints, but specify no trigger mechanism. Registration is the only trigger that starts a rollout — no MWAA DAG; the schedule and alert triggers in the Execution model below only move an already-started rollout along. Retraining cadence lives upstream: `09_monitoring.md` owns the triggers, the training pipeline executes them, and this job only reacts to the registration that results.

Task structure

Five tasks. Tasks 3 and 4 loop: each pass through them is one rollout state (fleet load at 0%, shadow, canary ramp — A/B nested inside the ramp when the policy enables it, `06_validation_promotion.md`).

1. Claim — sets the registry's challenger alias on the newly registered version. There's no evaluation here: that already happened in training (`05_training.md`), before the version was allowed to register.
2. Export — publish the ONNX artifact (named per the manifest's `onnx.file`; convention `model.onnx` — the rollout config's `checksums` map is keyed by on-disk filename, so the key must equal `onnx.file`, and the manifest's own filename is fixed as `manifest.json`) and `manifest.json` to an immutable S3 path, `<model_name>/v<version>/`. Training (`05_training.md`) already produced and parity-checked this artifact as part of its own run; this task publishes that output, it does not re-run training or redo the parity check.
3. Advance — write the next rollout config revision. A rollout state is nothing more than a particular config content; advancing means writing a new revision with the next state's fields.
4. Read out — read the per-arm tables built from decision rows and matured labels and produce a verdict: continue to the next state, hold, or abort. Readout logic: `06_validation_promotion.md`.
5. Promote or roll back — on a full pass through every state, move the champion and previous aliases and write the final config. A promotion under the `proxy` evidence mode also stamps a `trailing_until` tag on the model version, which keeps the scheduled passes judging it until the full window matures

(`06_validation_promotion.md`). On abort at any state, roll back instead: write a config pointing at the previous version.

Gate 2, before final promotion, is fixed by `06_validation_promotion.md` for every policy. Gate 1, before the first live exposure (shadow to canary), is a policy parameter — off by default. Either way, an active gate holds the readout pass until the model version carries the approval tag (Execution model, below).

The writes

The job is the only writer of both touchpoints. No other process writes them, including by hand.

Artifact store — one write per version, at export (task 2). The S3 path is immutable once written: never overwritten, never deleted. A bad version is abandoned, not erased; the rollout config simply stops pointing at it.

Rollout config — one write per advance (task 3) or per promote/rollback (task 5). Every write:

- increases `revision`. The lib reacts only to a revision change, so a write that does not bump it is invisible to serving.
- stamps `updated_at` and `updated_by` (this job is the only writer, so these are the audit trail).
- validates before writing: the referenced artifact exists at its S3 path, its checksums match the bytes actually there, the config is well formed. A failed validation aborts the write and the previous revision keeps serving.
- carries real checksums (`sha256:<hex digest>`; the lib's verifier compares the full string, so the format is enforced in practice even while the config schema itself is still draft-for-review) computed from the published files, never placeholder values.
- also checks that every hot feature the referenced manifest declares already has live keys in Redis. Not yet a contracted serving-side requirement: the serving repo lists a deploy-time check of a manifest's declared fields as a still-open need, and this implements the hot-feature half of it. It is also the handoff point with `04_features.md`'s feature-lifecycle rule: a manifest must never go live referencing a feature whose hot-feature job isn't running yet.

Both writes land in the same installation the Bidding Agent runs in (`00_overview.md`'s environments: dev and prod, each its own S3 prefix and config path). No cross-account hop; the job needs write access scoped to the environment it is running for.

Rollback

Rollback is task 5 taking the abort branch: write a config whose champion points at the previous version, with a new, higher revision. The previous version's artifact is still at its immutable S3 path, never deleted, so rollback never re-exports anything, it only rewrites the pointer. Every Bidding Agent process picks it up on its next poll of the config.

Where this runs

A Databricks job, not an MWAA DAG. MWAA orchestrates the data, feature, and training pipelines that feed into registration; registration is the handoff point where MWAA's responsibility ends and this job's begins.

Execution model

A Lakeflow deployment job is an ordinary multi-task Databricks job — the same kind of job run the training pipeline executes as (though this one is standing, the pipelines are submitted per run, `08_orchestration_repos.md`), with the same compute, environments (the Asset Bundle's wheel and dependencies), and permissions — plus one special property: it is attached to the registered model, so a new version registering triggers a run with the model name and version as parameters.

The five tasks are the logical structure. Execution is one job definition with two branches, chosen by a head condition task on the trigger type, because a rollout spans days of bake time and no run sleeps through them:

- registration-triggered branch — claim, export, advance (the first config revision); then the run exits.
- scheduled branch — one readout task: read the config state, the model-version tags, and the pre-aggregated per-arm tables; then write the next revision, write the rollback, or exit doing nothing. A third trigger, monitoring alerts, runs this branch's abort path immediately, so a guardrail breach never waits for the schedule.

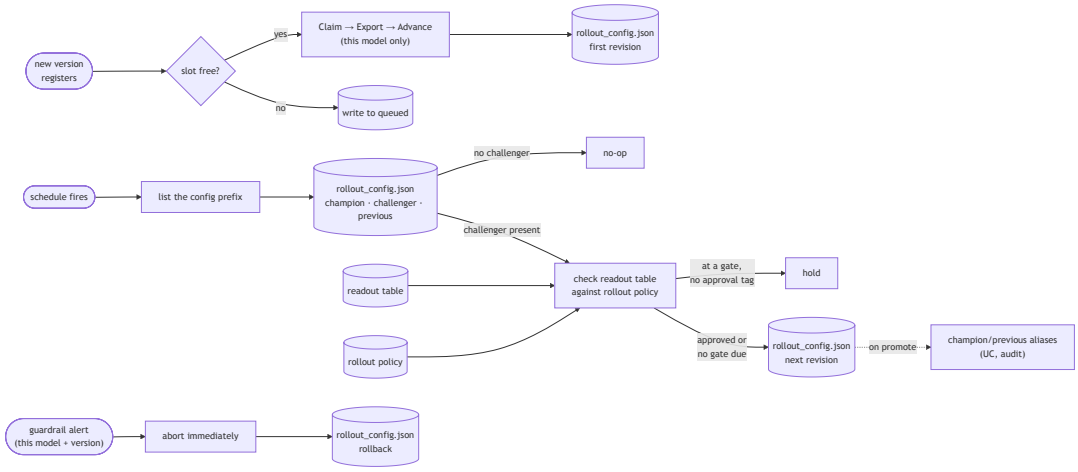
Compute is declared per task, and every task here is light: registry reads/writes, a file copy, a config write, or a query against pre-aggregated tables. No task runs inference at the model's own scale — that work happens in training. Serverless or a smallest-instance node covers every task regardless of model size; an idle pass costs seconds.

State and idempotency: no state store exists beyond the touchpoints — the rollout config's current content is the rollout state, and the model version's tags carry the approvals. Every pass is idempotent; each state transition happens in its own run; a pass advances at most one state.

Approvals are asynchronous: the approver sets a Unity Catalog tag on the model version (`approved_canary`, `approved_promote`); a pass holds, notifying the approver, until the tag exists. Who may set tags is registry access control, so approval is audited for free.

The schedule is a standing property of the job definition, never paused or mutated at runtime (the bundle stays the source of truth). Termination is logical: a terminal config has no challenger section, so later passes no-op — with one exception: a model version carrying an active `trailing_until` tag is still judged by each pass (the trailing confirmation after a proxy-mode promotion; breach writes the rollback, expiry ends the judging). One schedule serves every model's rollout — the pass scans all slots' configs and the trailing tags — while the registration trigger stays per model version.

Slot discovery for the scheduled branch: list the known config prefix — one `rollout_config.json` per model slot, sibling to that slot's `v<version>/` artifact folders.

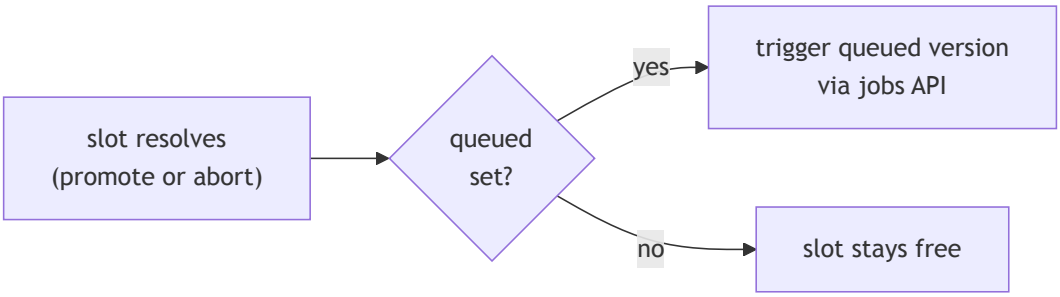


Concurrency and queueing

Only one challenger per model slot. Two things can make it busy when a new version registers: an experiment's A/B freezing promotion for days while scheduled or alert-triggered retrains keep happening underneath (`06_validation_promotion.md`), or the fast policy itself being slower than the retrain cadence even with no A/B involved.

Either way, the slot-free check above is the mechanism: busy means the new version's identity and registration time are written to `queued` on that slot's `rollout_config.json`, and the run exits without running claim or export — there's no free slot for either task to act on yet. `queued` holds at most one version, not a growing backlog: a new registration while the slot is still busy overwrites whatever was there, it doesn't add to it.

When the slot resolves (promote or abort), the scheduled pass checks `queued` and, if set, triggers that version's registration-branch work through the jobs API — the same self-trigger pattern the alert branch already uses.



Superseding happens the moment a newer version registers, not when the slot finally resolves: if N+1 is queued and N+2 registers while the slot is still busy, N+2 overwrites N+1 right then — N+1 is gone well before the slot ever resolves, not filtered out afterward. It never aborts a version that's already running its claim/export/advance or

sitting through shadow and canary — only “latest wins” among whoever is still waiting. Aborting in-flight work instead would livelock a slot whose retrain cadence outpaces its own bake time: nothing would ever finish.

Open questions

1. Queue race condition: if two versions register for the same slot near-simultaneously, both could read the config, both see the slot as free, and both try to claim it. The write path already increases `revision` and validates before writing (The writes, above) — needs confirming that this actually rejects the loser and forces a retry, not just that revisions happen to increase under the assumption only one writer ever runs at a time.

Nothing else open. The environment question (one definition vs one job per environment) is settled by `08_orchestration_repos.md`: one definition, instantiated per environment by Asset Bundle targets.

08 — Orchestration and repos

How code becomes running pipelines: the two repos, how pipelines are defined and submitted, environments, CI/CD, and testing. Mechanics only — what each pipeline does lives in its own doc.

The two repos

The split follows one principle: the workload repo is the production line, the DAG repo holds everything that says what gets built on it.

- Workload repo — pipeline-agnostic machinery: the versioned Python package (shared feature layer, cleaned ETL, label build, feature materialization, training pipeline stages, evaluation and readout logic — plus the config schemas those workloads consume), published as a wheel per release. It contains no job definitions and nothing model-specific, with one exception: the standing deployment job (below). Test fixtures are fixtures, not definitions.
- DAG repo — all definitions: a small framework (job-spec builders, parameter parsing — it renders Databricks run specs as data and never imports the workload package) and one folder per pipeline holding the DAG file, the pipeline YAML (parameters used at DAG rendering: schedule, compute tier, `package_version`), the model configuration the training run consumes, and the rollout policy the run passes through to the registry. Everything that defines Model A lives in its folder; adding Model B is a new folder, no workload change.

The MWAA parse-time environment decides the split on its own. The scheduler imports every DAG file continuously, in its own constrained Python environment: heavy imports slow every parse and every development loop, a stray workload import can break the scheduler's view of all DAGs, and changing MWAA's dependencies is a slow, risky environment update. ML code is dependency-heavy and large — the wrong tenant for that runtime. The workload package is never imported at parse time; its only import into the DAG repo is the config-schema subpackage, in CI (below).

Layouts:

```
dag-repo/                              workload-repo/
  framework/                             src/<package>/
    job_specs.py                           features/     (shared layer)
    params.py                              etl/          (cleaned)
    env.py                                 labels/
  pipelines/                               training/     (pipeline stages)
    training_pixel_purchase/               evaluation/   (gate + readout
logic)
      dag.py                               ingestion/    (decision-log ingest
      pipeline.yaml                          + readout-table build)
      training.yaml  (model                monitoring/   (metrics + trigger
        configuration)                       evaluation)
      rollout.yaml   (policy)              schemas/      (config schemas,
light
    training_pixel_page_view/                import path: no ML deps)
    cleaned_etl/                         resources/
    labels/
    feature_materialization/               deployment_job.yml
    hot_features/                        databricks.yml  (bundle targets)
    decision_ingest/                     tests/  (incl. fixture configs)
    monitoring/     (per-model thresholds
      in its YAML)
    backfill/
  tests/
  requirements.txt  (minimal, pinned, rarely changes)
```

Pipelines are submitted, not pre-created

The framework renders each run's full Databricks job spec at trigger time — tasks, compute from the pipeline YAML's tier, the wheel pinned to the YAML's `package_version` — and submits it through the Airflow Databricks provider (submit and wait for terminal state). No standing job objects exist for the pipelines; a pipeline is its folder.

Version coupling is explicit: a run executes exactly the package version its pipeline pins, so deploying a new workload release changes nothing until a pipeline opts in. One model can adopt a new version while another stays. The inter-repo contract is therefore the workload entry-point signatures and config schemas, versioned with the package itself; changing them is a release plus a per-pipeline YAML bump — two reviews by design, no deploy-ordering window.

DAG	Trigger / cadence	Workload entry points	Depends on
cleaned ETL	inbound cadence per dataset (hourly/daily tiers)	etl	curated data arrival (sensor)
labels	daily, after attribution lands	label build	cleaned attribution + impressions
feature materialization	per bucket cadence	materialization	cleaned partitions
hot features	~10 min	hot-feature compute	cleaned (freshness floor)
training (one per model)	`09_monitoring.md`'s triggers + manual	training pipeline stages	matured labels + feature tables
decision-log ingest + readout tables	~30 min (proposal, `09_monitoring.md`)	ingestion and readout workloads	decision-log landing zone
monitoring	daily, after ingest and label maturation (`09_monitoring.md`)	monitoring workloads (metrics + trigger evaluation)	decisions, readout tables, matured labels
backfill	manual, parametrized (workloads, range)	the affected workloads, in dependency order	—

The backfill DAG exists for the events that rewrite history: the client's attribution reprocessing (rebuild cleaned attribution → labels → downstream for the affected range, `02_datasets.md`), a new feature definition version needing its history materialized (`04_features.md`), or a fix in our own logic. It runs the same workloads as the scheduled DAGs — reruns and backfills are the same idempotent partition overwrite, only the range differs — but chains them across pipelines for one range as a single operation.

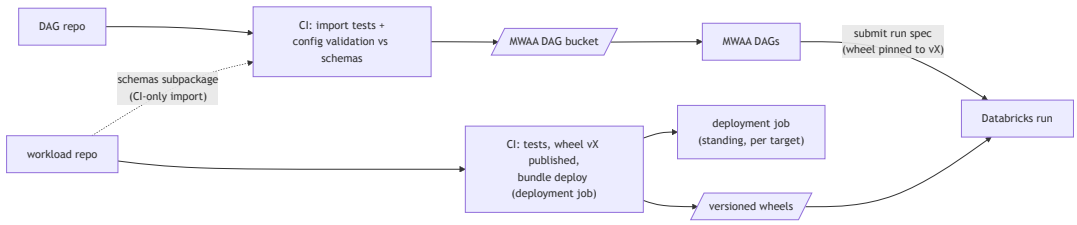
Pipeline contents per doc: cleaned layer `02_datasets.md`, labels `03_labels.md`, materialization and hot features `04_features.md`, training `05_training.md`, decision ingest with the readout-table build, and monitoring `09_monitoring.md`. The materialization, hot-feature, decision-ingest, and monitoring pipelines are single DAGs parametrized over the models: per-model ownership lives in the data (each model's key prefixes, `derived.feature_<model>_*`, and `derived.readout_<model>` tables), not in DAG multiplicity.

Configs and how they reach their consumers

- Definitions live in the DAG repo (per-pipeline YAMLs); their schemas live in the workload package, because the code that consumes a config owns its shape. The model configuration's contents are `05_training.md`'s; the rollout policy's (evidence mode, gate thresholds, bake times, traffic percentages, guardrail bounds) are `06_validation_promotion.md`'s. The rollout policy defines how the rollout works (the deployment job's input, read from the registry); the rollout config is the rollout's current state (the serving touchpoint the job writes to S3).
- The DAG passes the workload YAMLs to the run; the run validates them against its own schemas at start.
- The training run registers the resolved model configuration and rollout policy with the model version (`05_training.md`). Runtime consumers that are not DAG-triggered — the deployment job — read them from the registry, never from the DAG repo.

The deployment job

The one standing Databricks job, created from the one bundle in this repo (the bundle is the job's definition files, not the job itself). It is factory machinery: generic, model-agnostic, its per-model inputs arriving from the registry — so it belongs to the workload repo. `resources/deployment_job.yml` defines the job: its two execution branches, picked per run by the trigger (registration → claim, export, first advance; schedule → one readout pass that advances, holds, or aborts — `07_deployment_job.md`, Execution model), the per-task compute (every task is light — no task runs inference at the model's own scale, that happens in training), and the schedule. A third trigger exists at runtime: a monitoring alert (`09_monitoring.md`) starts a run of this same job through the jobs API, executing the abort path immediately instead of waiting for the next scheduled pass. `databricks.yml` declares the dev and prod targets. The workload repo's CI deploys it with the bundle per target; it runs the wheel its release shipped, which is correct for machinery. One definition, instantiated per environment by the bundle targets settles `07_deployment_job.md`'s environment question. Attaching it to a newly created model slot happens once, at the slot's first registration, from the training pipeline.



Environments

dev and prod. Working assumption: both live in the same AWS account — separate S3 prefixes, possibly one MWAA instance per environment, and separate Databricks environments; whether prod ends up in its own account is open. The design does not hinge on it: every environment parameter (catalog `ml_dev` / `ml_prod`, S3 prefixes, config paths, workspace) comes from the framework's environment config and the bundle targets, so an account split later changes deployment targets and the read grants the client extends, not the design. Pipeline YAMLs may pin different `package_version` per environment, so prod stays on a known release while dev tracks the latest. The deployment job is instantiated per environment by its bundle targets. Whether dev gets its own MWAA instance folds into the standing client question of reusing the client's MWAA versus running our own.

CI/CD

- Workload repo: lint + unit tests (including the light-import check: importing `schemas` must pull no ML dependencies) → build the wheel → publish it versioned (lean: a Unity Catalog volume) → bundle deploy of the deployment job to dev → smoke run on fixtures → prod on main.

- DAG repo: DAG import tests (parse must succeed without the workload package installed), config validation of every pipeline's YAMLs against the workload package's schemas (a CI-only import of the schema subpackage — never at DAG parse time), dependency assertions → sync to the environment's MWAA bucket. The bucket and requirements mechanics assume our own MWAA instance (the directed working assumption); under reuse of the client's instance those objects belong to their Terraform and our CI targets change accordingly.
- Secrets: one service principal per environment for Databricks and Unity Catalog; one MWAA-to-Databricks connection per environment; no secrets in either repo.

Testing

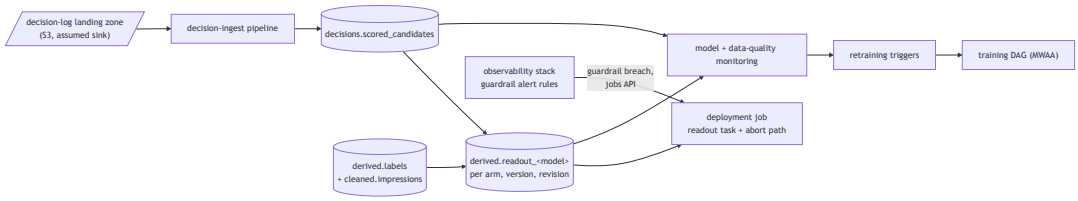
- Workload units: feature definitions against golden vectors (including fixed-point key-rendering cases, `04_features.md`); label-build join and maturity edges; calibration math (the closed-form base-rate correction has an exact expected output); config schema validation against fixtures.
- Integration, in dev: each workload submitted against `ml_dev` fixtures; the training pipeline end to end on a small snapshot — the export parity gate executes inside it by construction.
- DAG repo: parse tests, config-vs-schema validation, dependency assertions. No business logic lives there to test.

Open questions

1. MWAA: reuse the client's instance or run our own — the standing client question; decides where DAG code deploys and whether dev gets its own instance.
2. CI tooling: the client's core-app practice (CodePipeline) versus GitHub Actions; includes where versioned wheels are published (Unity Catalog volume is the lean).
3. Dev data: fixtures only, or a sampled slice of the curated datasets — gated on the privacy compliance review (`PLAN.md` open question 4) and the DE lane's access mechanics.
4. Repo hosting: the client's organization or ours.
5. Account topology for dev versus prod: same account (the working assumption) or separate accounts. Low design impact — it moves deployment targets, plus the cross-account read grants the client would need to extend to the new account — but tracked until settled.

09 — Monitoring and decision-log ingestion

The loop-closing doc: ingest what serving emits, measure models and pipelines, and decide when training runs again. Everything this doc triggers is specified elsewhere — the training pipeline in `05_training.md`, the verdict logic in `06_validation_promotion.md`, the deployment job in `07_deployment_job.md`, the scheduling in `08_orchestration_repos.md`. This doc owns the datasets built from decision rows, the metrics, the alerts, and the triggers.



Decision-log ingestion

Rows land in the S3 landing zone (which sink the Bidding Agent wires the lib's logger to is the serving side's open choice; S3 is the working assumption). The decision-ingest pipeline — scheduled and defined like every other pipeline (`08_orchestration_repos.md`), every 30 minutes as the proposal — loads them into `decisions.scored_candidates` (`02_datasets.md`): typed, deduplicated, partitioned by scored-at date. The cadence keeps canary and shadow readouts fresh during rollouts. Rows join served impressions and labels by (`request_id`, `candidate_id`).

Retention: decision rows are kept for the replay and readout horizon — long enough to replay against the longest attribution window plus the rollout's trailing confirmation. Proposal: 3 months, sized against real volumes before prod (`02_datasets.md` defers the number here).

Per-arm readout tables

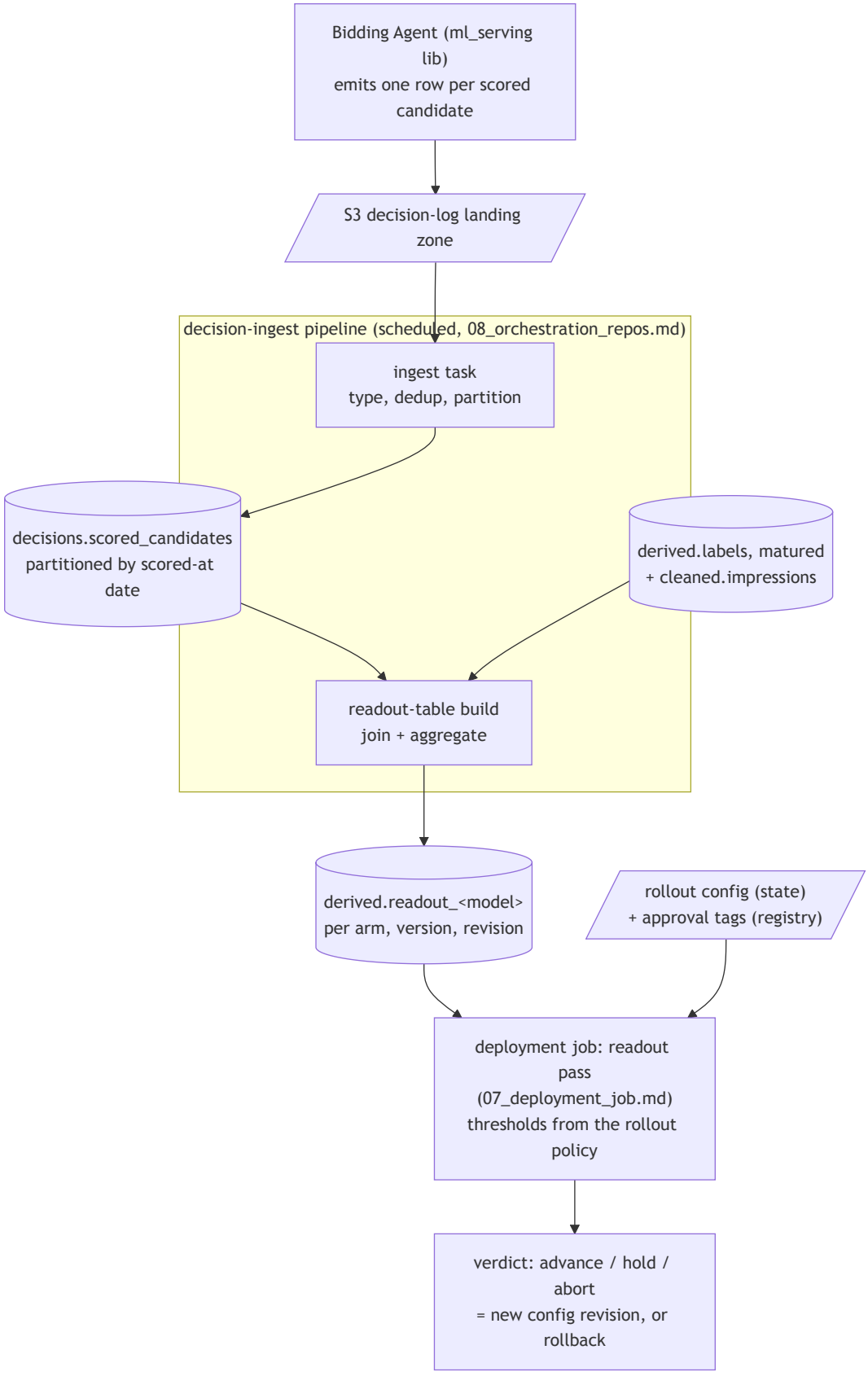
`derived.readout_<model>` — aggregates per model, version, arm, and config revision, computed from decision rows joined with matured labels: row counts, score-distribution statistics, realized conversion rate per window, predicted-versus-realized calibration by decile, and the observed arm shares the sample-ratio check reads.

Two steps share the word "readout". Naming them precisely:

Name	Runs where	Reads	Writes / produ
readout-table build	task in the decision-ingest pipeline (this doc, scheduled per `08_orchestration_repos.md`)	`decisions.scored_candidates`, matured labels, impressions	`derived.re` (incremental re partitions)
readout pass	the deployment job (`07_deployment_job.md`)	`derived.readout_<model>`, rollout-config state, approval tags	verdict — adva a config write, (`06_valida`

The trailing confirmation is the readout pass judging a promoted model whose version carries an active `trailing_until` tag (stamped by the promote task after a proxy-mode promotion): the full window's maturing labels, rollback armed, until the tag expires.

The detailed dataflow — who produces what, where it lands, who reads it (cylinders are tables, parallelograms are S3 objects, rectangles are jobs or tasks):

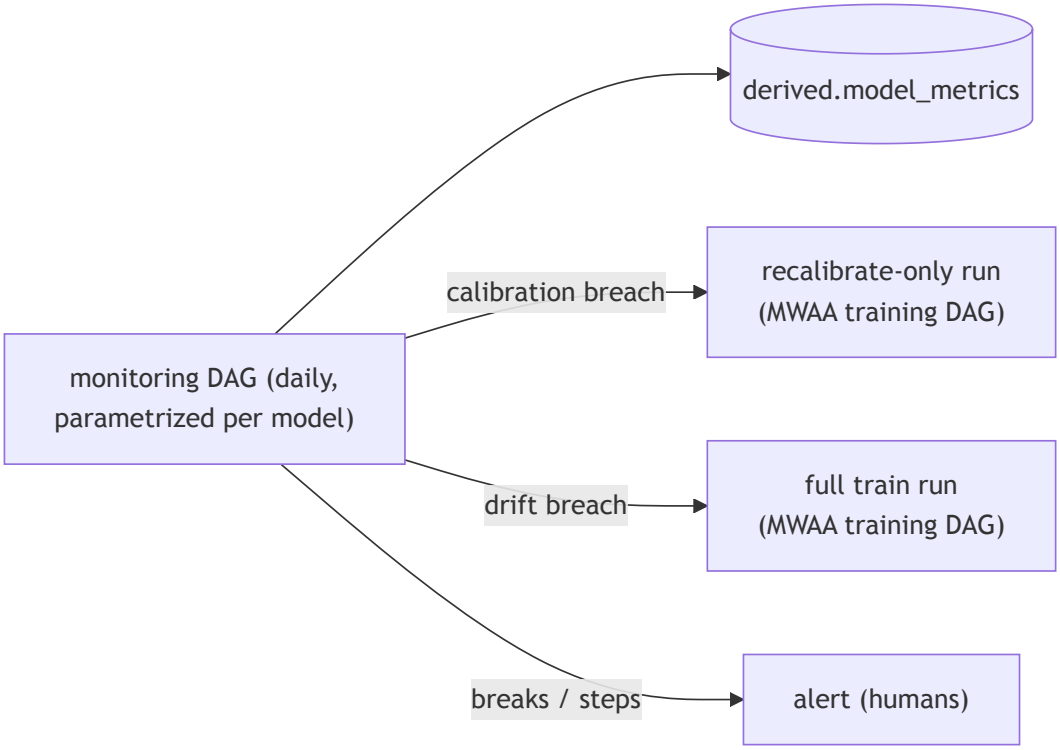


Defaulted-feature rates come from the scored-versus-default flag each decision row carries: the share of candidates scored with at least one defaulted feature, per arm.

Model monitoring

One monitoring DAG, parametrized over the models like the hot-feature pipelines (`08_orchestration_repos.md`), scheduled daily after ingest and label maturation. Per model it computes metric rows into `derived.model_metrics` — one row per model, metric, cut, and day — then evaluates each metric against its thresholds and fires the mapped action. Thresholds live in the monitoring pipeline's workload YAML, one entry per model (a per-model definition, so the DAG repo holds it; `08_orchestration_repos.md`); reference distributions come from the training run's logged artifacts (`05_training.md`).

Metric	Computed from	Method	Threshold rule → action
calibration	champion decision rows joined with matured labels, per window	reliability by decile + expected calibration error; cuts: advertiser tier, campaign, time slice	error beyond bound for N consecutive days → recalibrate-only run; persistently high per-advertiser-tier error → flag for advertiser-layer activation (human review, `05_training.md`)
score drift	live score distribution from decision rows vs. the training run's logged holdout distribution	population stability index (a bucketed divergence score)	sustained drift beyond bound → a full train run + alert
input drift	logged inputs and hot-feature values vs. the training run's logged input distributions	per-feature divergence + null shares	sustained drift on high-importance features → a full train run; a single broken feature → alert the feature pipeline, no retrain
defaulted-feature rate	the scored-versus-default flag per candidate	share of candidates defaulted, per model over time	step change → alert (a feature job or the agent's fields drifted from the manifest); never a retrain trigger
cost per action	readout tables, per goal	realized cost against the goal, as an expected-performance signal (never a guarantee of observed CPA per period — the client's framing)	reported only; business signal, not a trigger



Data-quality monitoring

The checks the other docs assigned, owned here: each lands in `derived.data_quality` (one row per check, dataset, and day). Recording is distributed, judging is centralized: each data-producing pipeline (cleaned ETL, label build, materialization, hot features) records its own numbers as the last step of its run — the table's "Written by" column — while the monitoring DAG computes the two checks that are no pipeline's byproduct and alone evaluates every alert rule. Those two guard the feed itself: freshness catches data not arriving (newest partition age vs. the expected cadence, per dataset), reconciliation catches data arriving wrong (our cleaned totals vs. the client's own hourly rollups, on normalized units). The training pipeline and the deployment job write nothing here. No separate data-quality DAG exists.

Check	Written by	Computed from	Rule → action
inbound schema conformance	cleaned ETL, per run	inbound columns and types vs. the expected schema per curated dataset	mismatch → alert; the affected partitions do not load
inbound-feed freshness (`01_data_access.md`)	monitoring job, daily	latest partition age per curated dataset vs. the client's proposed cadence	age beyond cadence + margin → alert
orphan-conversion rate (`03_labels.md`)	label build, per run	conversions whose impression key matches no impression row, per day	rising rate → alert: the impressions and attribution feeds disagree about what was served
null-`ad_id` share (`03_labels.md`)	label build, per run	attribution rows with null `ad_id`, per day	tracked and reported with rollups; alert on step change
label coverage per goal	label build, per run	matured impressions and positive labels per goal and window, per day	collapse in volume or positives → alert: measurement or feed problem upstream of any model
reconciliation (`02_datasets.md`)	monitoring job, daily	cleaned totals vs. the aggregate performance dataset, on normalized units	divergence beyond tolerance → alert: a cleaned pipeline or the inbound feed is wrong
feature-table staleness (`04_features.md`)	materialization pipeline, per run	latest bucket age per model and grain	lag beyond the bucket cadence → alert
Redis keyspace health (`04_features.md`)	hot-feature pipeline, per run	key count per model prefix vs. expected combinations	shrinking count → priority alert: a hot-feature job is failing and serving is silently defaulting

Pipeline alerting

DAG and job failures, run-duration anomalies, and every freshness or data-quality breach above alerts; hot-feature job failures alert with priority, because their failure mode is silent defaulting in serving. Routing lean, not decided: one pane consistent with the client's stack (CloudWatch alarms, Slack routing, Datadog APM), fed by Databricks and MWAA alerting — the bidder's monitoring will likely be managed independently of the client's own infrastructure monitoring (the client's infra docs say "likely," not a firm expectation), so the choice is ours; Databricks-native SQL alerts are the fallback shape if a second pane is acceptable.

Rollout guardrails on serving health have a named producer: alert rules we define in the observability stack, whose action starts the deployment job's abort path through the jobs API (`07_deployment_job.md`). They never route through the daily monitoring job.

Retraining triggers

This doc owns when training runs; the pipeline executes (`05_training.md`). Two kinds: a scheduled cadence per model, set in the pipeline YAML (`08_orchestration_repos.md`), and the degradation triggers from the model-monitoring table above (calibration breach → recalibrate-only; drift breach → a full train run, or tune and train per the model's tuning spec; threshold values set at EDA). Mechanically, the monitoring workload records the verdict and the monitoring DAG triggers the model's training DAG in MWAA. Guardrail breaches during a rollout take the faster path: they start the deployment job's abort branch directly through the jobs API (`07_deployment_job.md`), never waiting for a readout schedule.

Bidder response dataset

The outbound dataset the client proposed, with a stated need for visibility into how the bidder decided. Assembled as a derived table plus an export job on the same pipeline mechanics: our `decisions.scored_candidates` provides the model side (scores, model version, timestamps, candidate identity, with the client and DSP ids as output columns via the identity triplets); the Bid Decision service's own logs provide the bid side (bid price, bid/no-bid, no-bid reason) and the agent supplies latency and the selected strategy; the shared request and candidate ids stitch them. Assembly crosses lanes, since the agent side owns its fields, and the client's required field list and cadence are still their call.

Open questions

1. The decision-log sink: which store the Bidding Agent wires the lib's logger to. S3 is the working assumption everything above builds on.
2. Alert routing: the single pane for the bidder account's monitoring (likely managed independently of the client's own infra monitoring) — lean stated above.
3. Drift and degradation thresholds, and the minimum volumes behind them — set at EDA.
4. Bidder response dataset: required fields, cadence, and consumers — the client's call, plus the bid-decision lane's fields.
5. The decisions retention horizon — the 3-month proposal needs real volumes (readout and monitoring tables follow the derived layer's retention rule, `02_datasets.md`).

Part II — Annex

Client Custom Bidding Agent — Discovery Summary

Purpose: Durable, single-file knowledge base for the client custom-bidding-agent project, ML-lane focused. Consolidates client-provided facts and our team's analysis so context survives across sessions and people. **Date:** 2026-09-22 · **Updated:** 2026-10-20 (design_docs set complete, 00–09; final whole-design audit) · **Status:** living summary

> **How to read this file.** Two top-level parts are kept strictly separate: **Part 1 — Client-provided facts** (what the client docs state) and **Part 2 — Our analysis & working decisions** (what our team has reasoned or leaned, from the ML Lane Discovery Report and ADRs). Never treat Part 2 as client-confirmed. Open questions to the client live in `design_docs/OPEN_QUESTIONS.md` (§2.4 below is a historical trace only).

Part 1 — Client-provided facts

Source: the client documents under `docs/batch_01/md/` (initial doc set) and `docs/batch_02/md/client_reports/` (2026-10 background reports, §1.9). These are statements from the client, not our conclusions.

1.1 Project overview

- The client runs a self-service platform for premium TV/streaming advertising (per the project brief, which is marked outdated; no current doc restates it). It is building a **custom bidding agent** for real-time bid decisioning, to gain more granular, dynamic bid control than the ad server's current process allows.
- The agent is **co-located with the ad server bidding infrastructure** and speaks the ad server's **protobuf interface**. It ingests a bid request + the array of eligible line items/creatives (no line-item retrieval needed), decides, and returns a response.
- **Bidding formula (performance advertisers):**

  ```
  Final Bid = λ (pacing discount) × Expected Value
  Expected Value = Max CPA × P(Goal Action | Impression)
  ```

  - **Max CPA** — advertiser-defined max cost per desired action, collected in the client's Ads Manager.
  - **P(Goal Action | Impression)** — predicted probability the impression yields the desired action (the client's ML models).
  - **λ (pacing lambda)** — per-line-item discount from comparing expected vs. actual spend-to-date; recomputed each pacing cycle; does **not** need to run in the synchronous bid path.
- **Awareness vs. performance:** Awareness campaigns keep CPM goals, modeled as `P(Impression | Impression) = 1`, so only λ is dynamic. **Today the client does not offer cost-per-action bidding — all advertisers are effectively "awareness."** This project enables performance bidding.
- **Future:** "auto bid" (no Max CPA; bid modulated entirely by λ, equivalent to a very high Max CPA) enabled via a line-item flag once the services exist.

1.2 The four services and request flow

The agent is designed as logically separate components so pacing, prediction, and bidding evolve independently (the SW-team benchmark doc stresses these are logical boundaries, **not** mandated separate deployments):

#	Service	Responsibility
2.1	**Bid Ingestor / Processor**	Entry point. Deserialize/validate the ad server protobuf, extract candidates + request context, orchestrate downstream calls, manage lifecycle within latency, handle invalid requests, log/observe.
2.2	**Discount Pacing (λ)**	Periodically compute per-line-item λ from expected vs. actual spend-to-date; expose latest λ via low-latency lookup; configurable refresh; safe behavior when λ is stale/missing. Does not necessarily need to run in the synchronous request path (client wording).
2.3	**ML Prediction (Expected Value)**	Return `P(Action)` per candidate line item. New models/goals must be addable without changing bid-processing architecture. Latency approaches suggested: in-memory inference, precomputed features, low-latency serving, or precomputed value lookups.
2.4	**Bid Decision / Response**	Combine Max CPA, `P(Action)`, and λ into `Final Bid = λ × Max CPA × P(Action)`; select candidate(s); build a valid `BidAgentResponse`.

- **Request flow (our reading, per the SW-team benchmark doc's Approach 1 — not a client statement):** the ad server → Ingestor → (Pacing lookup ‖ ML prediction) → Bid Decision → response. (The client's end-to-end diagram is `image1.png`, which we cannot render; its labels are client-infra components, not the agent's internal flow.)
- **External contract (per the SW-team benchmark doc):** the ad server sends **HTTP POST** with a protobuf `bid.BidAgentRequest`; agent returns **HTTP 200** with `bid.BidAgentResponse` or **HTTP 204** (no bid). The 10 ms end-to-end latency is treated there as a strict response timeout (the client-authored docs state it as an *initial target*); same-AWS-region deployment is the ad server's recommendation. Processing budget must leave room for transport. Bid prices are **integer micros per impression** (e.g., 0.8 × $20 × 0.001 = $0.016 = 16,000 micros = $16 CPM).
- Schemas are the DSP's OpenRTB extensions (`request.proto`, `adcandidate.proto`). Candidates carry line-item/creative IDs, bidding settings, frequency caps, fees.

1.3 Monolith vs. multi-service benchmark (SW team)

The SW team proposed benchmarking two topologies (pick the simplest that meets latency/throughput/reliability with headroom): - **Approach 1 — Distributed Go microservices** over gRPC (processor fans out to pacing + prediction; bid calc stays with processor for the comparison). Compare unary vs. streaming; measure serialization, network, queueing. - **Approach 2 — Modular in-process Go app** (one deployable, four components behind an orchestrator, no internal network). Four execution variants: (A) sequential, (B) per-request goroutines, (C) persistent worker pools, (D) direct pacing snapshot read + inference worker pool. - The initial prediction interface **should return probabilities explicitly**; EV and bid are computed once downstream. - Benchmark payloads are official DSP fixtures (`sample_bid_request_1.txt`), maintained as protobuf text, compiled to binary. Three bundled payloads are **banner examples (1–2 candidates, ~1.1–2.1 KB binary)** — explicitly **not verified client production traffic**. Payload matrix scales candidates (1→500) and creatives; video profiles planned but need real samples.

1.4 Data & labels

- **Prediction target uses attribution, not raw pixels** (client: measurement and attribution data provides the labels and outcomes for model training). The positive/negative construction — positive = served impression matched to an attributed conversion within the window, negative = served impression with none — is **our** label design (ADR-0004), resting on the unconfirmed wins-only-universe assumption.
- **Web attribution → the web-attribution provider**; **in-app → MMP (mobile measurement partner) trackers**. Windows: **7 / 14 / 30 day** tables documented in the batch-01 data models; the batch-02 ML report states a **1-day** variant is also available (“one/fourteen/thirty day are available”, plus its `PIXEL_PURCHASE_1_day` example and its train-on-1-day question) — 1-day table to confirm with the client.
- **Late-maturing labels:** attribution windows stay open for days, so recent data is only partially labeled and conversions arrive after delivery.
- **Goals (scope doc, with the client’s own hedges): Pixel Purchase** (“likely the top priority goal we’d want to ship”), **Page View** (“likely the p0 goal” if purchase isn’t possible), **App Install** (out of MVP scope), plus future optimization events. (Batch-02 later makes Page View and Purchase co-priorities — see §1.9 and contradiction #5.)
- **Key tables:**
  - `gold_kochava_{seven,fourteen,thirty}_day_attribution` — web-attributed conversions (query/reporting layer; keys: conversion_id, enhanced_pixel_id, ad_id, adaccount_id).
  - `mobile_app_event_{7,14,30}_day_attribution` — app-attributed conversions (keys: conversion_id, app_tracker_id, ad_id; has revenue, impression_id).
  - `raw_performance_data` / `win_logs` — raw DSP delivery/win-log data in S3 (auction, bid, cost, supply, device, geo, content, model metadata). **Downstream integer IDs, not client UUIDs.**
  - `performance_report_pipeline_merged` — hourly aggregate per `ad_id × hour` (impressions, clicks, conversions, spend, ctr/cpa/cvr, video quartiles).

    - Supporting: `kochava_conversion_data`, `attributed_conversion_data`, `raw_attributed_conversion_data`, `app_events_conversion_data`, `raw_performance_data_kochava`, raw `kochava`.
- **Identity mapping (critical):** the client stores its own **UUIDs**; for objects that also exist in the DSP, the DSP ID goes in the client's `external_id` and the client UUID goes in the DSP's `alt_id`. Raw DSP logs carry **integer** `campaign_id`, `line_item_id`, `creative_id`, `account_id` — **must be mapped** via `external_id`, never treated as UUIDs. Unified bidder datasets must expose client UUID, DSP ID, and raw-log ID as separate, clearly named fields.
- **Entity hierarchy:** organization → adaccount → campaign → **adset (≈ DSP line item, the primary delivery config object: bid, budget, dates, pacing, targeting, tracker links)** → ad → creative → media. `campaign.objective_type` known values **include** {web_conversions, awareness, app_promotion} (the data models say "includes", not "only" — same hedge as `adset_type` below); the `adset_type` enum currently **includes video**. Web measurement uses **enhanced_pixel** (now via the web-attribution provider); app uses **app_tracker** (MMP). Admin `adset_delivery_override` takes precedence over defaults.

1.5 Infrastructure & environment

- **The client runs in AWS** inside a private VPC (VPC/subnets managed by the ad server, imported into client Terraform).
- **Client data/warehouse stack:** Postgres (RDS, source of truth for config/entity state), S3 (raw + semi-processed), Athena (query), Apache Iceberg (table format), Glue Catalog + Lake Formation (governance), **Glue + MWAA/Airflow** for data jobs.
- **App compute:** ECS/Fargate; deploys via CodePipeline/CodeBuild/CodeDeploy/ECR; all IaC in Terraform (`dev` / `stg` / `prod` roots).
- **Secrets/keys:** AWS Secrets Manager + KMS. **Observability:** CloudWatch alarms, Slack alerts, **Datadog APM**.
- **The bidder most likely runs in its own AWS account** (the client's wording is "most likely"; another client doc frames the bidder instead as a Terraform-ownership boundary that may consume shared VPC/Postgres/Athena — the topology is not final), consuming client data **across an account boundary** either way. The client remains source of truth for config, eligibility, attribution, and warehouse outputs; the bidder owns model execution, decisioning, experimentation. Cross-account sharing via cross-account IAM, Lake Formation sharing, S3 replication/export, event streams, or APIs. **Bidder infra expectation (the client's infra doc): a dedicated EKS cluster, ECR, pipeline, IAM roles, security group, private subnets, CloudWatch, Secrets Manager.** RDS access = add bidder SG to RDS ingress on 5432; Athena access = workgroup + Glue reads + S3 results + Lake Formation grants.
- **Bidder should consume curated client outputs, not raw internal tables.**
  Proposed dataset split: delivery-control feed, impression-level training dataset, attribution-labels dataset, aggregate performance dataset, bidder-response dataset.

- **Data-cadence tiers (the client's proposed expectations for the curated feeds, not live cadences):** | Tier | Contents | |—|—| | Immediate / event-driven | eligibility changes: pause, archive, spend lock, delivery disable, budget stops, critical override changes | | Frequent / near-real-time | active delivery settings, bid config, targeting eligibility, deals, creative eligibility | | Hourly | delivery metrics, spend, impressions, clicks, win rate, bid/win diagnostics | | Daily | attributed conversions, finalized reporting, training labels, feature snapshots | | Weekly / backfill | large historical training sets, attribution reprocessing, feature rebuilds, model experiment datasets |

  The delivery-control feed's target cadence and implementation are explicitly still to be determined (client).

1.6 Compliance

- **AI-use annex (Sigma-issued, contractual — not a client statement):** AI systems not provided by the client's parent company may not be used for the work without prior **written approval from Sigma**, and AI systems in use must be disclosed. The annex is in the doc set (`docs/batch_01/md/`, the AI-use annex document; filename still carries the unredacted parent-company name — pending rename). Practical effect: likely rules out external ML SaaS unless approved.
- Data models flag many **privacy-sensitive** fields (`platform_device_ifa` / `platform_device_idfa`, `ip_address` / `ipv6_address` / `impression_ip_address`, `hhid`, `user_id`, `user_email`, device ids) and consent signals (`is_gdpr`, `is_gdpr_consented`, `gdpr_consent_string`, `us_privacy`, `dnt`, `lmt`) requiring privacy-aware handling and retention rules.

1.7 Non-functional / success criteria (client-stated)

- < 10ms end-to-end HTTP; horizontal scaling; health checks; controlled deploy/rollback.
- Monitor: total/processed requests, response rate, e2e + per-component latency, errors/timeouts, candidates evaluated, `P(Action)`/λ/Max CPA/EV/final bid used, win/loss, spend/pacing; down-sampled decision-level logging to reconstruct any bid.
- Phase-1 success = reliably ingest/parse the ad server requests, evaluate candidates, obtain a probability/value signal, retrieve λ, combine with Max CPA, compute final bid, respond within latency, deploy/scale reliably, and upgrade components independently.

1.8 Process

- **Technical debt process:** logged in a register (Description / Identified / Owner / Estimate / Priority / Status / Jira ID / Comments), reviewed at least biweekly (Sigma DSP team: Monday calls), promoted to Jira when scheduled. The provided register is an empty template.

1.9 Batch-02 client reports (2026-10)

Source: `docs/batch_02/md/client_reports/`. Two background reports from the client. The ML one is our lane; the pacing one is reference only (pacing is another lane).

ML Service Background (2026-10-11)

- **Bidding strategies via two new ad set fields** (do not exist yet in the ad set data model): `bidding_strategy` ∈ {`max_cpm`, `goal_cpa`, `lowest_cost` (= "auto bid")} and `optimization_goal` — the action the advertiser wants to drive, hence which model to consult (the client lists eventual support for `impressions`, `PIXEL_PAGE_VIEW`, `PIXEL_PURCHASE`, and `app_installs`). Together they determine whether inference is needed, which model is used, and how `max_bid` is interpreted.
- **Expected value per impression, by strategy** (`Final Bid = λ × EV` for all three):

Strategy	EV per impression	Model use
`max_cpm`	`max_bid / 1,000`	none — effective goal is impressions, `P = 1`; deterministic non-ML path
`goal_cpa`	`max_bid × P(Action)`	model selected by `optimization_goal`
`lowest_cost`	`Internal CPA Value × P(Action)`	same goal-specific model; no advertiser CPA limit — a high internal CPA value scales the bid and λ finds the level that spends the budget

All equations are per impression; a CPM bid representation multiplies by 1,000. Units must stay consistent across config, model output, EV calculation, and bid response. The goal-CPA behavior is an expected-performance objective, "not a guarantee of observed CPA in every reporting period"; the evaluation window and the treatment of fees "remain to be defined".

- **Attribution window may become advertiser-facing config:** e.g. `optimization_goal = PIXEL_PURCHASE_7_day` vs. `PIXEL_PURCHASE_14_day`, or a separate ad set column. Directly reshapes the ADR-0004 framing: the window may be a product dimension rather than one canonical choice.
- **Initial model priorities: Page View AND Purchase**, both supporting goal_cpa and lowest_cost. New goals (app installs/purchases, lead generation) must be addable later without replacing the core bidding architecture.
- **Sparse-signal advertisers are a hard requirement:** the approach cannot depend on every advertiser having enough conversions for an advertiser-specific model; it must serve new/low-signal advertisers and incorporate advertiser-specific information as it arrives. Shared models vs. an advertiser-specific calibration layer is explicitly open for discussion.
- **ML interface left open:** the service "does not necessarily need to produce the full EV but must at least provide the goal p(Action)" — probability vs. EV vs. both is a technical design decision; applying pacing and building the response stays with the Bidding Service. Consistent with ADR-0001.

- **Latency framing:** the <10 ms target is the full real-time path, not ML alone; ML fits its allocated portion. In-memory inference, precomputed features, low-latency serving, and precomputed lookups are named as options, not a selected architecture.
- **Required logging:** prediction, value input, expected value, and λ used per bid, so any decision can be reconstructed. Models and new optimization goals must evolve without redesigning request processing.
- **Training datasets named:** auctions and bid logs in `vir-prod-uaadsmanager-reporting-db` (`auction_us_{east,west}`, `bid_logs_us_{east,west}`); win logs in `ua-data-jobs-prod-reporting.win_logs` (batched) and `vir-prod-uaadsmanager-reporting-db.silver_win_logs_stream` (streamed); pixel conversions `silver_kochava_events` + `gold_kochava_seven_day_attribution` ("one/fourteen/thirty day are available" — the 1-day variant is new vs. the batch-01 data models); app conversions `bronze_mobile_app_events` + `gold_app_events_seven_day_attribution` (same window note). The report names only the two databases; the query engine is unstated (Athena is our inference from the client's stack).
- **Client's open ML questions:** which attribution windows to offer per goal and how to model them (separate models per window vs. train on 1-day and up-calibrate); sparse-signal/new-advertiser handling; calibration and evaluation per goal incl. launch-readiness criteria; features available at bid time vs. precomputed, the ML share of the latency budget, multi-candidate evaluation, and fallback behavior; whether the interface returns probability, EV, or both, and how models are tested/deployed/rolled back.

Pacing Service Background (2026-10-07) — reference only, other lane

- Defines discount pacing **λ** (bid-price multiplier) vs. probabilistic pacing **α** (auction inclusion rate); proposes a hybrid start where the client Pacing Service computes λ (even curves first) and the ad server's Pacing-as-a-Service keeps α. Kept only for the λ semantics used in the bid formula; the pacer is not our scope.

Part 2 — Our analysis & working decisions

Source: `reports/discovery_report/ML_Lane_Discovery_Report.md` (Rodrigo Cunha, 2026-09-22, delivered and frozen), the `adr/` set, and the team's testing docs (`docs/batch_02/md/testing/`, §2.7). These are our team's reasoning and leans — not client-confirmed. Open questions are in `design_docs/OPEN_QUESTIONS.md`.

2.1 What we own

The **ML lane only**: the service that returns a **calibrated** `P(Goal Action | Impression)` per candidate within the ML slice of the 10ms budget, plus the offline pipeline that builds/refreshes it. Two halves: - **Online (hot path, ms):** assemble features, batch-score all candidates, return a probability each; fallback on missing model/feature. - **Offline (batch):** join served impressions to attributed conversions, train + calibrate, evaluate, version in a registry, publish model + precomputed features to serving.

MVP goal: credible calibrated probabilities for the two initial goals (Page View, Purchase), extensible to more goals/models without redesign.

2.2 Working leans (Discovery Report §2)

Area	Lean
Target & labels	**Page View and Purchase both MVP goals** (batch-02); **one global model per goal, single window each for now**, extending to one model per (goal, window) once the client's offered windows firm up; advertisers as features (no per-advertiser models — client's sparse-signal constraint). Window may become advertiser-facing config; up-calibration from a shorter window kept as a conditional alternative (ADR-0004).
Features	Mostly **request-derived** features (free at bid time) + a small set of high-value **precomputed** features from a low-latency store; bid-time work = lookups only. Cold-start needs defaults.
Serving architecture	**ONNX tabular GBDT on CPU** for MVP, **embedded in the Go agent** — settled on the serving side as a pure Go library the agent imports (no server mode, no gRPC/HTTP listener). The agent's own monolith-vs-multi-service topology is still the SW-team benchmark's question, but it no longer affects the ML embedding choice. **ONNX regardless**, so serving can move (embed → Python → Triton) without retraining. **Triton deferred** to a later deep-model/GPU phase.
Training infra	**Databricks (client-confirmed, direct client knowledge — postdates the batch-01 docs):** workloads/compute, Delta as table format, Unity Catalog. Orchestration via MWAA/Airflow; whether we reuse the client's instance or run our own is open. Offline platform design in `design_docs/`.
Tracking & registry	**MLflow with Unity Catalog on Databricks** (experiment tracking + model registry). Datadog is for serving monitoring, not tracking.
Calibration (cross-cutting)	**Required** — output is multiplied by dollars, so it must be a true probability. Two corrections fit offline on held-out data and **baked into the served ONNX artifact**: (1) **base-rate correction** to undo negative-downsampling class shift (closed-form; skip if not downsampling); (2) **post-hoc calibrator** (Platt / isotonic) applied after. Calibrator is **versioned with the model as a matched pair**. Track live via reliability diagram / ECE on an attribution lag. Future, not MVP: **per-advertiser calibration** for data-rich advertisers — calibration parameters keyed by advertiser id as an in-graph lookup (Gather + global default row), fit as a group-by with shrinkage toward the global fit, gated by a minimum-volume eligibility threshold; per-advertiser calibration error is a standard evaluation cut from MVP on, to detect when this layer earns its place. Per-window calibrators use the same mechanism on the artifact axis (same trees, different calibrator per window).

Area	Lean
Data quality	Lightweight offline validation (schema, nulls/coverage, label coverage, attribution freshness, drift, train/serve parity) + serving-side monitoring of feature nulls and prediction distribution.
Lifecycle	Configurable retraining cadence; training windows must account for late-maturing labels; publish via **offline eval gate → serving**, with rollback; shadow/canary/A-B designed in (design_docs 06), activation per rollout policy; orchestrate with MWAA; monitor in Datadog.
Fallback	On missing model/feature/timeout/cold-start, return a safe default probability so the agent still answers in 10ms.

2.3 Assumptions until told otherwise (Discovery Report §3)

1. MVP model = tabular GBDT on CPU, exported to ONNX.
2. Two initial goals (Page View, Purchase), a single window per goal for now; one model per (goal, window) as the end state.
3. Labels from the client's curated attributed-conversion datasets (web-attribution provider for web / MMP for app), not raw tables.
4. **ML service returns `P(Action)`; the `× Max CPA` multiply lives in the Bid Decision service** (see discrepancy below).
5. Serving embedded in the Go agent as a pure library (ADR-0002, amended 2026-10-19) — settled regardless of the agent's own monolith-vs-multi-service topology, which the SW-team benchmark still decides.
6. Training and data workloads run on Databricks (client-confirmed); orchestration via MWAA/Airflow.
7. Tracking/registry = MLflow with Unity Catalog on Databricks; Datadog for serving monitoring.
8. Scores calibrated post-hoc (Platt/isotonic) + base-rate correction if downsampling; calibrator baked into the served artifact.

2.4 Open questions (moved)

The maintained lists now live with the design: `design_docs/OPEN_QUESTIONS.md` is the consolidated register of every open question (client / EDA-gated / team / serving-side, each with its sources), and `design_docs/CLIENT_QUESTIONS.md` is the client-facing subset, phrased to send. The delivered report's Q1–Q16 (2026-09-22) remains a frozen snapshot with its own numbering. Refer to questions **by topic, never by number**. The historical list below is kept only for the settled-item trace:

- **Target & labels:** (1) label volumes per goal and window — both goals (Page View, Purchase) ship per batch-02; volumes decide each goal's windows, not which goal goes first (§1.9); (2) which windows will be offered per goal (possibly advertiser-selectable, §1.9), confirming the 1-day variant the batch-02 report names, and

whether the attribution method (click- vs. view-through) can be exposed at all; (3) real attribution coverage / handling of unmatched/privacy-limited conversions? (4) existing CVR baselines to calibrate against?

- **Data access & features:** (5) sanitized real bid-request samples for premium video/TV (current samples are banner)? (6) mechanism + cadence for the bidder account to consume client data — partially answered: batch-02 names the source tables (§1.9); the cross-account mechanism stays open, now owned by the DE lane; (7) achievable feature freshness per category — partially answered: the cadence tiers are published (§1.5); still open whether a fresher-than-hourly boundary feed is achievable; (8) Max CPA at bid time — **answered by batch-02**: the ad set's existing `max_bid`, read from ad set config and reinterpreted per `bidding_strategy` (CPA under goal_cpa, §1.9).
- **Serving & latency:** (9) expected/peak throughput and candidates-per-request distribution — team working numbers exist (1M req/s, 10 candidates/request, §2.7) pending client confirmation; (10) latency budget for the ML step within the 10ms envelope — settled: a firm ~3ms p99 (our target, CLAUDE.md "Our lane"); (11) Go monolith or multi-service — internal (SW-team benchmark), not a client ask; kept for completeness; (12) GPU available, or CPU-only for MVP?
- **ML interface:** (13) probability vs. expected value — the client leaves it explicitly open (§1.9); our proposal is probability only (ADR-0001, contradiction #1), to present for their confirmation.
- **Tooling/infra/compliance:** (14)(15) training infra and registry — **settled: Databricks** (workloads, MLflow tracking, Unity Catalog registry; direct client knowledge); (16) reuse the client's MWAA or stand up our own — still open; (17) monitoring standard — ours to decide (Datadog lean), not a client blocker.
- **Lifecycle:** (18) retraining cadence and pre-serve validation — ours to decide after EDA, not a client blocker; (19) the goal-CPA evaluation window and fee treatment — the client's ML report marks both as remaining to be defined; we need them to compute the cost-per-action readouts (design_docs 06).

2.5 Dependencies & risks

- **Cross-account data access** — critical path; nothing trains/serves without it. Owned by the DE lane; the ML lane consumes the five curated datasets as a given.
- **Attribution definition** (windows per goal, volumes, method) — defines the prediction target; blocks the window choice per goal (label build itself is parametrized).
- **Real traffic samples** — needed to validate feature availability/latency on real video payloads (current fixtures are banner).
- **Compliance** (AI-use annex) — the annex constrains AI-system use for the work; tooling itself is settled (Databricks). Confirm annex scope against the signed contract.
- **Agent topology** (monolith vs. multi-service) — owned by the SW-team benchmark; the ML embedding choice is settled (pure library) and no longer depends on it.

2.6 ADR status

The `adr/` set uses MADR-lite (see `adr/template.md`, `adr/README.md`):

- **ADR-0000** — record architecture decisions (Accepted).

- **ADR-0001** — ML service returns calibrated `P(Action)`, not expected value (Accepted; amended 2026-10-15 — reaffirmed as our proposal to the client's now-explicitly-open interface question, with per-candidate response metadata and caller-side goal routing).
- **ADR-0002** — MVP serving = ONNX tabular GBDT on CPU, embedded in Go as a pure library; Triton deferred (Accepted; amended 2026-10-19 — embedding settled as a pure Go library with no server mode, superseding the earlier embed-vs-co-located fork; the agent's own topology no longer affects this choice).
- **ADR-0003** — calibration mandatory, versioned with the model as a matched pair, baked into the served artifact (Accepted; amended 2026-10-15 — the calibration component may be parameterized: per-window variants and a future per-advertiser lookup with shrinkage, still in-artifact).
- **ADR-0004** — attribution window + method defining the training label (**Proposed, blocker**; reframed 2026-10-15 — goal × window model granularity as default, window possibly advertiser-facing config, up-calibration as conditional alternative; method still pending the client).
- **ADR-0005** — cross-boundary data-access / data-engineering ownership (**Proposed, blocker**; reassigned 2026-10-15 to the **Data Engineering lane** — ML treats the five curated datasets as a given and states requirements).
- **ADR-0006** — the served artifact is the self-contained pipeline (preproc → model → calibration in one ONNX graph) (Accepted, discovery — revisable).

2.7 Testing lane (Olena, 2026-10)

Team-authored testing docs under `docs/batch_02/md/testing/` — ours, not client-confirmed:

- **Test strategy (draft):** TypeScript monorepo (pnpm) with Playwright (component/integration, E2E, UI) + k6 (performance), Allure reporting, Prometheus + Grafana for performance runs. CI tiers: per-PR smoke, nightly regression, weekly performance runs on a dedicated production-representative environment. E2E uses a deterministic ad-server emulator; integration testing against the real ad server is owned by the ad-server vendor's integration team. Rollout validation (shadow, canary, blue/green, progressive) is shared TA/ML/SRE.
- **Performance targets captured there (team-stated, to confirm with client):** 1M bid requests/sec throughput; E2E p99 ≤ 10 ms; **ML prediction p99 ≤ 3 ms**; 10 candidates/request as the initial assumption. Method: 1 warm-up + 10 measured runs, p99 evaluated per run, never averaged across runs.
- **AI test harness (vision + ROI):** a Claude Code-based agentic layer on top of the test-automation solution (requirements analysis → test design → implementation → execution/triage), build phases H0–H8 with human approval gates throughout. ROI model v2.0 (draft): reference case breaks even ~8.4 months, ROI 100% ~16 months, sensitive mainly to triage volume. Effort estimates in `testing_estimate.md`.

Team roster

Per the task brief.

Name	Role	Focus
Rodrigo Cunha	Principal ML Engineer	Owner of the ML lane (this summary, discovery report, ADRs)
Matheus Bergmann	Senior ML Engineer	ML lane
Illia Slobozhanin	Principal Data Engineer	Data engineering
Augusto Pietroski	Senior SW Developer	Ingestor / pacing benchmarks
Yuri Mednis	Lead SW Developer	Software
Olena Ozerova	Principal QA	Quality

Artifacts (current)

Artifact	Location	Contents
ML Lane Discovery Report	`reports/discovery_report/ML_Lane_Discovery_Report.md`	Discovery-level M lane plan, deliver 2026-09-22 and frozen: design ar A–H with leans, assumptions, clie questions, risks.
This summary	`DISCOVERY_SUMMARY.md`	Consolidated clie facts + our analys (this file).
ADR set	`adr/`	ADR-0000 meta 0001–0006 (statu §2.6); `README.` (index), `template.md`
Offline platform design	`design_docs/`	Design docs for t offline ML platfor — the set is complete (00–09) overview, data access, datasets labels, features, training, validation/promo deployment job, orchestration/rep monitoring.
Client questions	`design_docs/CLIENT_QUESTIONS.md`	The top-priority questions for the client, self-contai and sendable.
ML serving lib	github.com/cunha-sigma/ml_serving	The MVP Go serv library (ONNX art + manifest, rollou config, hot-featu mirror, decision logs). Externalize repo; no local co
Serving benchmark	github.com/cunha-sigma/ml_serving_bench	Frozen serving-latency benchma (engines `leave` ONNX Runtime, stage-placement arrangements, serving report). Externalized repo local copy.

Client source docs live under `docs/batch_01/` (initial set: `Documents/`, AI annex, tech-debt process; converted Markdown in `batch_01/md/`) and `docs/batch_02/` (2026-10: `client_reports/` + Olena's `testing/`; converted Markdown in `batch_02/md/`). The arch-diagram image is `batch_01/md/Documents/_media/media/image1.png` (not machine-readable here).

Contradictions & gaps noticed

1. **ML output = probability (our proposal — see ADR-0001, amended):** the client docs' formulas have the Bid Decision service apply `× Max CPA`, but the client's own prose names the service in EV terms (scope-doc heading "ML Prediction Service (Expected Value formulation for Ad Ranking)"; the batch-02 report's "providing the expected-value input"), and batch-02 leaves probability-vs-EV explicitly open ("must at least provide the goal p(Action)"). ADR-0001 (probability only) is our answer to that open question, to present to the client.
2. **Serving-topology tension:** the client's infra doc prescribes a **dedicated EKS (Kubernetes) cluster** for the bidder, while our ML lean is an **embedded/co-located Go** serving path (ONNX on CPU) to minimize hops, with Triton/GPU deferred. These are reconcilable (EKS can host a Go service): the ML side is settled as a pure Go library embedded in the agent (ml_serving; no server mode), validated by our serving benchmark (ml_serving_bench) and recommended as the all-in-ONNX artifact embedded in Go (ADR-0006). What remains open is the agent's own runtime shape (monolith vs. multi-service — the SW-team benchmark question).
3. **Banner samples vs. premium-video product:** all provided fixtures/payloads are **banner** examples (from the DSP tool), explicitly not client production traffic; the product is **premium TV/video** (the `adset_type` enum currently includes video). We cannot validate feature availability or latency on representative payloads yet (the real-traffic-samples question, `design_docs/OPEN_QUESTIONS.md`).
4. **AI-use annex (Sigma-issued, now in the doc set):** present in `docs/batch_01/md/` (filename still carries the unredacted parent-company name — pending rename). It restricts using AI systems not provided by the client's parent company for the work without written approval; it does **not** govern ordinary tooling choices (e.g., MLflow). Confirm its scope against the signed contract.
5. **Goal-priority drift across client docs:** the batch-01 scope doc framed **Purchase as likely top priority with Page View as the likely fallback**; the batch-02 ML report (§1.9) prioritizes **Page View and Purchase together** as the initial models. Both goals ship; label volumes decide each goal's windows, not which goal goes first.
6. **Empty tech-debt register:** process is defined but the register is a blank template — no logged items yet.

ADR-0000: Record architecture decisions for this project

Status: Accepted · **Date:** 2026-09-22 · **Deciders:** Rodrigo Cunha

Context. client custom-bidding-agent is in discovery; decisions formed from client docs and analysis need a durable record that survives across sessions and team members.

Decision. Keep a lightweight ADR set (MADR-lite), ML-lane focused, recording only load-bearing decisions — those costly to reverse or needed to keep downstream work coherent. New ADRs added by agreement.

Consequences. - Single place for why the system is shaped as it is; update the `README.md` index on every add or status change. - Non-critical detail stays in working docs (`DISCOVERY_SUMMARY.md`, `scratchpad.md`), not here.

ADR-0001: ML service returns calibrated P(Action), not expected value

Status: Accepted · **Date:** 2026-09-22 · **Amended:** 2026-10-15 (batch-02 ML Service Background) · **Deciders:** Rodrigo Cunha (ML lane)

Context. `Final Bid = λ × Max CPA × P(Goal Action | Impression)`. Open question: does the ML service return the raw probability `P`, or the expected value `Max CPA × P`? The batch-01 docs' formulas have the bid-decision service apply `× Max CPA` and `× λ` downstream, while the client's own prose names the service in EV terms (the scope doc's section heading is "ML Prediction Service (Expected Value formulation for Ad Ranking)"). The batch-02 ML Service Background then reopens the question explicitly: the service "does not necessarily need to produce the full EV but must at least provide the goal p(Action)", and probability vs. EV vs. both "remains a technical design decision". It also introduces per-candidate heterogeneity: each candidate ad set carries its own `bidding_strategy` and `optimization_goal`, and the value multiplied depends on the strategy (`max_bid` as CPA for `goal_cpa`, a system-assigned internal CPA value for `lowest_cost`, `max_bid/1000` with no model for `max_cpm`).

Decision. The ML service returns one **calibrated probability `P(Goal Action | Impression)` per candidate**, plus score metadata; `× value` and `× λ` apply downstream in the bid-decision service. This is our proposal to the client's open interface question. The bidding agent groups candidates by goal (later by goal and window), calls the serving lib once per model slot (slots named by goal), and owns the strategy→value interpretation and the multiply.

Why. - The value side is bidding policy, not prediction: `max_bid` interpretation is strategy-dependent, and the `lowest_cost` internal CPA value is a budget-delivery knob owned by another lane. Returning EV would import both into the model boundary. - A probability stream is monitorable for calibration against realized conversion rates; an EV stream mixes model drift with config changes and confounds experiment-arm comparisons. - λ and the final bid live outside ML in every option, so decision reconstruction requires a log join key regardless; moving the multiply solves nothing there. - A uniform response schema holds across strategies and goals: a new goal is a new model slot, no interface change.

Consequences. - The returned number must be a true probability → calibration mandatory (ADR-0003). - The per-candidate response carries: calibrated probability, the goal (and window, if windows become ad-set config) the score answers, model name/version/arm/revision, and a scored-versus-default status flag so the caller can distinguish a real score from a fallback. - Decision-log rows need the request/candidate join key so probability rows join the agent's value/λ/bid rows (tracked in the serving repo; the client's reconstruction-logging requirement makes it firm). - Present to the client as our answer to their open interface question.

ADR-0002: MVP serving is ONNX tabular GBDT on CPU; Triton/GPU deferred

Status: Accepted · **Date:** 2026-09-22 · **Amended:** 2026-10-19 (embedding settled) · **Deciders:** Rodrigo Cunha (ML lane)

Context. The agent must answer within the ad server's 10 ms end-to-end timeout; the ML Prediction step's own slice is a firm ~3 ms p99 target. For tabular models inference is microseconds; the hot-path cost is hops, serialization, feature lookup, and runtime overhead — not the model math. Model family is open long-term (deep models remain possible).

Decision. MVP is a **tabular GBDT (LightGBM/XGBoost) exported to ONNX, served on CPU, embedded in the Go agent** as a pure Go library (ml_serving — no server mode; settled 2026-10-19, superseding the earlier embed-vs-co-located fork). **Triton deferred** to a later deep-model/GPU phase. ONNX is the format regardless, as a portability hedge.

Why. - Embedded in-process: best latency, no hop; as a library it fits whatever topology the agent takes. - Triton rejected for tabular: its hop/runtime dominate microsecond inference; earns its keep only for deep models on GPU.

Consequences. - Minimal serving infra for MVP; no separate inference fleet. ONNX keeps embed → Python → Triton open without retraining. - The agent's own topology (monolith vs. multi-service) remains the SW-team benchmark's question and no longer affects the ML embedding. Revisit for deep models/embeddings → GPU + Triton.

ADR-0003: Calibration is a versioned matched-pair artifact baked into the served model

Status: Accepted · **Date:** 2026-09-22 · **Amended:** 2026-10-15 (parameterized-calibration note) · **Deciders:** Rodrigo Cunha (ML lane)

Context. The service returns `P(Action)` (ADR-0001) multiplied by real dollars, so it must be a **true probability**, not a ranking score. Raw GBDT scores are uncalibrated (tree models are over-confident), and downsampling negatives (common for rare conversions) shifts the base rate and inflates scores.

Decision. Produce the calibrated probability with two offline steps, fit on a held-out temporally-split set with the true base rate, and **bake the result into the served ONNX artifact**: 1. **Base-rate correction** — only if negatives are downsampled; closed-form odds rescale by the negative keep-fraction. Skipped otherwise. 2. **Post-hoc calibrator** — Platt or isotonic, after the base-rate correction.

The calibrator is **versioned with the model as a matched pair** (a calibrator fit on vN is invalid for vN+1); model + calibrator (+ encoder) promote and roll back as one unit.

Why. Separate/unversioned calibrator risks serving a mismatched pair; no calibration distorts every bid.

Consequences. - Offline pipeline gains a calibration step and a dedicated held-out split; artifact bundle is model + calibrator (+ encoder). - Monitored live (reliability diagram / ECE), on a lag due to attribution windows. A cheaper **recalibrate-only** refresh (refit calibrator on recent data, reuse model) is permitted. - The calibration component may be **parameterized** without breaking the matched-pair rule, in two forms (2026-10-15): - **Per-window variants** — the same trees exported with a different calibrator per attribution window, each fit on that window's matured labels; one self-contained artifact per (goal, window) (ADR-0004). - **Per-advertiser rows (future, not MVP)** — calibration parameters as an in-graph lookup (advertiser id → params via Gather, with a global default row), fit as a group-by with shrinkage toward the global fit on the same held-out slice, gated by a minimum-volume eligibility threshold. Per-advertiser calibration error is a standard evaluation cut from MVP on, to detect when this layer earns its place. In both forms the table is fitted with the model, versioned with it, and baked at export; the artifact stays self-contained.

ADR-0004: Attribution window and method that defines the training label

Status: Proposed (blocker — needs client input) · **Date:** 2026-09-22 · **Amended:** 2026-10-15 (batch-02 ML Service Background) · **Deciders:** Rodrigo Cunha (ML lane); pending client

Context. The label for `P(Goal Action)` comes from attribution: a positive is a served impression matched to an attributed conversion within a window; a negative has none. The batch-01 data models document **7/14/30-day** window tables for the web-attribution provider (web) and MMP (in-app); the batch-02 ML report states a **1-day** variant is also available ("one/fourteen/thirty day are available", plus its `PIXEL_PURCHASE_1_day` example) — the 1-day table needs confirming with the client. Labels mature late, so recent impressions are under-labeled. The batch-02 ML report changed the shape of the question: the goals are enumerated (`PIXEL_PAGE_VIEW` and `PIXEL_PURCHASE` both initial; app events later), and the **window may become advertiser-facing configuration** — a goal-suffixed value (`PIXEL_PURCHASE_7_day`) or a separate ad set column. If so, the window is a product dimension, not a single canonical choice. The client also asks the modeling question directly: separate models per window, or train on a short window and up-calibrate?

Decision. Pending client input. Working lean, replacing the earlier single-canonical-window lean: - **Goal × window model granularity** as the expected end state: one model per (goal, window), each trained on its own window's labels. For now each goal ships on a single window, so slots are named per goal; when windows multiply, the routing key and slot naming extend to (goal, window) (ADR-0001). - **Up-calibration** (train on the shortest window, fit a per-window calibrator on matured long-window labels) stays on the table as the client-suggested alternative, not the default — their question is framed on the 1-day window ("train off a 1-day view through and then up-calibrate?"), which presupposes the 1-day table. Its legitimate triggers are bootstrap history poverty (maturity eating the training range early in the system's life) and freshness; its validity gate is a ranking-invariance experiment — the long-window conversion rate must be a stable monotone function of the short-window score across segments. Mechanism in ADR-0003. - **Method** (click- vs. view-through) unchanged: the client's window tables record no method; nothing to choose until the client exposes it.

Why. Open axes: which windows the client will actually offer per goal (now a product question, not just ours); label volumes per (goal, window); whether per-window rankings differ enough to require separate trees (likely goal-dependent — fast goals like page views may up-calibrate safely, laggy purchases likely not).

Consequences. - The question to the client sharpens: which windows will be offered per goal, and label volumes per (goal, window) — top client-meeting priority. - The label build stays parametrized over (goal, window); nothing blocks on the modeling choice itself. - Drives training-window design, late/under-labeled handling, and calibration ground truth per window.

ADR-0005: Cross-boundary data-access pattern (curated exports vs. direct Athena)

Status: Proposed (blocker — needs client input; owned by the Data Engineering lane) · **Date:** 2026-09-22 · **Amended:** 2026-10-15 (ownership reframe + batch-02 source inventory) · **Deciders:** Illia Slobozhanin (Data Eng lane), Rodrigo Cunha (ML lane, requirements side); pending client + infra lane

Context. The bidder most likely runs in its own AWS account (the client's docs say "most likely"; a competing doc frames it as a Terraform-ownership boundary inside shared infra instead) and must consume client data (training data + online features) across an account boundary either way. The docs conflict: the infra-and-data-context doc says use **curated, bidder-prepared datasets** (push/export); the infra doc prescribes **direct Athena + Glue + Lake Formation** grants (pull/query). The batch-02 ML report made the question concrete by naming the internal source tables (auctions and bid logs in `vir-prod-uaadsmanager-reporting-db`, east/west; batched `win_logs` and the web/app conversion and per-window attribution tables in `ua-data-jobs-prod-reporting`; a streamed `silver_win_logs_stream`) — whether that list is the menu the client curates from, or tables we query directly, is unresolved.

Decision (ownership, 2026-10-15). This decision belongs to the **Data Engineering lane** (team DE, not yet started), together with the curation mechanics it implies (regional unions, batched-vs-streamed reconciliation, cross-account mechanism, identity-mapping execution). The **ML lane treats the five curated datasets as a given** — whether the client delivers them or our DE curates them from the named sources is immaterial to the ML platform design. The ML lane's part is the **requirements contract** on those datasets: the impression-level dataset is the label universe (served impressions) and carries a key the attribution rows reference; the attribution dataset preserves per-window structure and `event_name`; the id triplets arrive filled; hot-feature inputs meet the freshness floor. Working lean on the pattern itself is unchanged: curated datasets for pipelines, direct Athena at most for exploration.

Why. - Curated exports (push): the client owns joins/identity mapping and cadence; stable contract, decoupled from the client schema; least ML-lane scope. Less flexible for ad-hoc needs. - Direct Athena/Lake Formation (pull): the consuming side owns the joins; flexible and always-current; couples to the client schema, heavier cross-account IAM, largest scope increase — now DE-lane scope, not ML-lane. - Hybrid: the client delivers core joined datasets; scoped direct access for exploration.

Consequences. - Still the critical path for training data and online features; drives IAM, feature freshness, schema-coupling risk. Escalation and resolution sit with the DE and infra lanes. - The ML-lane docs (`design_docs/`) design against the five curated datasets and record the requirements contract; they adapt to whichever mechanism the DE lane lands. - The batch-02 table list is recorded as the provenance inventory for the DE's work.

ADR-0006: Ship preprocessing, model, and calibration as one ONNX file

Status: Accepted (discovery — revisable) · **Date:** 2026-09-25 · **Deciders:** Rodrigo Cunha (ML lane)

Context. Scoring a candidate takes three steps: **preprocessing** (turn raw inputs into model features) → **model** → **calibration** (turn the score into a true probability, ADR-0003). Each step can either be packaged inside the ONNX file or hand-written in the Go service (ADR-0002). Hand-writing a step in Go risks it drifting from the training-time code (train/serve skew), and Go's ML tooling is thin.

Decision. Package **all three steps in one ONNX file** and run it inside the Go agent. The service stays thin: raw record in → probability out, running the model slot's own file (one slot bound to one model at construction), with no model-specific code. ONNX is the interface for every model.

Why. The serving benchmark (ml_serving_bench, laptop — cloud confirmation pending) shows bundling all three into one file costs only **~0.07 ms** over a model-only file. So it's effectively free, and it removes the drift risk because training and serving run the exact same code.

Consequences. - The offline pipeline must be able to export the model alone, or with preprocessing, or the full bundle — so we can change what's inside the file later without rebuilding training. - The service never computes stateful features; those are precomputed offline and fetched (see feature store). Anything ONNX can't express is precomputed offline too. - The pure-Go `leaves` engine can't run preprocessing/calibration inside the file, so it would force those back into Go — a mark against it versus ONNX Runtime. - Revisit if EDA or cloud numbers change the picture.